浙江大学第二十四届
大学生数学建模竞赛

2026 年 5 月 8 日—5 月 18 日

团队编号 182

题　　目　　A　　B

（在所选题目上打勾）

	参赛队员 1	参赛队员 2	参赛队员 3
姓名	黎艺皓	王欣雨	郑涵予
学号	3250100365	3250103525	3250103520
院（系）	求是学院云峰学园	求是学院蓝田学园	求是学院丹青学园
专业	信息工程	电气工程及其自动化	统计学
手机	18105729580	18367258706	15268709858
Email	yihao_sirius@163.com	3250103525@zju.edu.cn	3250103520@zju.edu.cn

浙江大学本科生院

浙江大学数学建模实践基地

基于深度学习的透明饮料瓶残留液体识别模型与优化策略

摘要

透明塑料饮料瓶内残留液体的自动识别是垃圾分类与自动分拣中的关键技术难题，空瓶与少液瓶的视觉差异极度微弱（液面线仅占 3-5 像素），且受瓶体姿态、光照、标签遮挡等多重因素干扰。

本文基于自建数据集，建立了一套融合迁移学习、数据增强与端到端检测分级的数学模型，有效解决了小样本、难分类、类别不平衡及泛化能力提升等挑战：

针对任务一（数据集构建）：自行拍摄并整理共 340 张真实饮料瓶图片，涵盖 12 种瓶型、6 种液体类型、4 种含量等级（空、少量、中等、多量）以及多种姿态、背景和光照条件。使用 MakeSense 工具进行 YOLO 格式人工标注，按 8:2 分层划分为训练集（256 张）和验证集（64 张），并预留 20 张独立测试集。四类样本量基本均衡（各约 25%），为后续建模提供了可靠数据基础。

针对任务二（二分类基准与优化）：将四类标签映射为“有液体/无液体”二分类。采用预计算增强策略（CLAHE 边缘增强、不对称空间变体生成）[1]，缓存为.pt 文件以提升 GPU 利用率；基于 ResNet18 迁移学习构建分类器[2]，输入尺寸 288×288。损失函数采用 Focal Loss（$\gamma=2$，$\alpha_1=2.0$）聚焦难分样本[5]，优化器选用 SGD+Momentum，结合加权采样、Dropout(0.5)和权重衰减抑制过拟合[3][4]。在独立测试集（20 张）上，模型达到准确率 80%、精确率 100%、召回率 77.8%、特异度 100%、F1 分数 0.875，且对不同瓶型和姿态具有良好泛化能力。

针对任务三（自动定位与含量分级）：将模型推广为端到端 YOLO11n 四分类检测框架，同时输出瓶体边界框和含量类别（空/少量/中等/多量）。训练 30 轮，输入分辨率 640×640，采用 Mosaic、MixUp 等内置增强。测试集上检测成功率达 100%，平均 IoU 为 0.87；四分类总体准确率 70%，空瓶类精确率和召回率均为 100%，少量类召回率 62.5%。合并为二分类后准确率达 100%，验证了端到端方案的可行性。[6]

综上所述，本文提出的模型在小样本、难分类场景下实现了高精度液体残留识别，并成功扩展至自动定位与含量分级，为垃圾分类智能化提供了有效技术路径。

关键词：饮料瓶液体识别；小样本分类；ResNet；Focal Loss；YOLO；数据增强

一、 问题重述

1.1 问题背景

在垃圾分类、自动分拣、资源回收等场景中，需要判断透明或半透明塑料饮料瓶内是否残留液体。瓶体可带有瓶盖、标签、纹理或反光，但需存在可观察的透明区域。瓶内残留液体可以是清水、茶饮料、果汁、碳酸饮料等常见饮品，可能无色透明或者有色透明。每张图片中只包含一个饮料瓶，瓶体形状基本完整，没有严重挤压变形或大面积遮挡，瓶子的位置和姿态可以任意变化（竖立、横放或倾斜）。由于人工判断效率低且成本大，现需要建立自动识别模型。

该问题的核心难点在于：空瓶与少液瓶的视觉差异极度微弱。液面线（空气-液体界面）在低分辨率下仅占 3-5 个像素，且受瓶体姿态、光照条件、瓶体材质、标签遮挡等多重因素干扰。我方在获取标注图像后，需要建立数学模型，通过优化数据预处理、模型架构、损失函数及训练策略，使模型能够准确判断瓶内是否有液体，并追求在极限难度样本上的泛化能力最大化。

1.2 问题提出

基于上述背景，本研究需要建立数学模型，以最大化识别准确率为目标，解决以下四个递进的策略优化问题：

1.2.1 模型的建立与验证

给定已标注瓶体区域的图片数据集（共 320 张，原始标签分为空瓶、少量、中等、多量四类，各类样本量分别为 81、83、77、79，占比基本均衡），现合并为无液体（空瓶，81 张）与有液体（少量+中等+多量，239 张）两类。需建立基准模型判断瓶内是否有液体。模型输入为经边界框裁剪后的瓶体区域，输出为二元分类结果。需要确定合理的评价指标体系，包括验证集准确率（Val Acc）、召回率（Recall）、精确率（Precision）、特异度（Specificity）以及泛化置信度分布等指标进行综合评价。

1.2.2 小样本条件下的模型优化策略

数据集有效图片仅 320 张（无液 81 张，有液 239 张），而空瓶与少液瓶的视觉差异极为微弱——液面线在低分辨率下仅占 3-5 个像素，且受瓶体姿态、光照条件、瓶体材质、标签遮挡等多重因素干扰。在建立模型时，需研究适用于小样本、难分类任务的数据处理与模型优化策略。具体需要解决以下子问题：输入模态的选择（RGB 三通道与多模态输入的对比）、数据增强策略的设计（预计算与在线增强的分工，避免训练与测试分布不一致）、损失函数与训练策略的选择、主干网络容量的控制（在多

种 ResNet 架构中选择合适的参数量）、模型训练优化器的选择、正则化方案的配置（运用多种正则化技术的组合以抑制过拟合）、采样策略的设计以及推理阈值的确定。

二、问题分析

本文的核心任务是建立透明塑料饮料瓶内残留液体的自动识别模型，并在此基础上实现瓶体自动定位与液体含量分级。该问题本质上是融合了图像预处理、目标检测、小样本分类与多任务学习的复杂模式识别问题。总体解决思路是：首先构建包含瓶体标注与四类含量标签的数据集；然后针对“有无液体”二分类任务通过类别映射，训练模型；然后扩展能力，实现自动定位+液量自动识别。

根据任务要求，将问题分解为以下四个递进的处理进程：

2.1 数据准备与基础建模

采集得到数据集共 340 张标注图片，四类（空、少量、中等、多量）样本量基本均衡（各约 25%）。先分出 20 张作为测试集，剩余样本按比例分为训练集与验证集。在任务二中，四类标签需要进一步映射为二分类（空→无液体，少量/中等/多量→有液体），从而获得有液样本 239 张、无液样本 81 张，存在一定的类别不平衡（约 3:1）。

随后，对任务二建立基于迁移学习的二分类基准模型，以瓶体标注框裁剪后的区域为输入，输出归一化后是/否有液体的概率。评价指标包括准确率、精确率、召回率、特异度、F1 分数等，以全面衡量模型性能。

对任务三建立基于 YOLO11n 微调的定位与多头分类模型，输入包含单个瓶体的图片，输出瓶体位置与其归属不同类别的置信度。

2.2 小样本条件下的模型优化

有效样本仅 320 张，而空瓶与少量样本的视觉差异极为微弱——液面线在低分辨率下仅占 3-5 个像素，且受瓶体姿态、光照、标签遮挡等干扰。需要考虑输入模态（引入专门的边缘增强通道）、数据增强策略（通过变换扩展训练可用样本数同时尽量不加剧过拟合）、主干网络容量（ResNet18/34/50）以及正则化方案（Dropout+权重衰减）四个维度进行优化。核心目标是避免过拟合，提升模型对难分样本（尤其是少量类）的辨别能力。

2.3 问题三

此问题为类别不平衡与难分样本的针对性优化。原始四类中“少量”样本（83 张）与“空”样本（81 张）数量相近，但视觉特征高度重叠，容易相互误判。需设计专门的

处理策略：包括损失函数的选择（如 Focal Loss 以聚焦难分样本）、优化器的对比（SGD vs AdamW）、采样策略（加权随机采样或类别平衡采样）以及推理阈值的动态调整（降低有液阈值以提升召回）。通过分析混淆矩阵中空与少量的误判情况，可评估优化效果。

2.4 模型推广

任务三要求实现自动定位+液体含量分级系统。可以采用定位+检测的双阶段处理，并通过 TTA（推理时对单图做 5 个随机变换采样，平均预测）；或者直接微调 YOLO 基座模型统一实现定位与多分类任务：需解决的关键技术包括：检测与分类的级联推理效率、少量与中等的边界模糊处理、以及跨瓶型/姿态/光照的泛化能力验证。最终在测试集上评估分级准确率及检测定位精度。

三、模型假设

基于透明塑料饮料瓶内残留液体识别问题的实际挑战，为建立标准化、可控的数学模型，我们提出以下合理假设：

3.1 瓶体区域标注准确且完整。

假设用于训练和测试的图片中，人工标注的瓶体边界框完全包含整个瓶身（包括瓶盖与瓶底），且未包含过多的无关背景区域。标注类别（空瓶/有液）基于真实液体残留情况判定，忽略因拍摄角度导致的液面视觉偏差。

3.2 瓶身透明区域足以观察液体。

假设所有图片中的饮料瓶均存在至少一个可观察的透明或半透明区域（如无标签覆盖的瓶体中部），且该区域能够清晰呈现液面线、液体颜色或瓶底残留痕迹。完全被不透明标签包裹或材质完全不透光的瓶子不在此次研究范围内。

3.3 图片质量满足基本识别要求。

假设所有图片由手机或摄像头拍摄，分辨率足够（如不低于 320×320 像素），瓶体整体清晰可见，无严重运动模糊、过度曝光或欠曝光。图片中仅包含一个饮料瓶，且瓶体无严重挤压变形或大面积遮挡（如被手、其他物体遮挡超过瓶体面积的 30%）。

3.4 液体类型与含量对建模的影响可忽略。

假设瓶内残留液体为常见饮品（清水、茶饮料、果汁、碳酸饮料等），其物理化学特性（如黏度、表面张力）不影响液面线的视觉表现。液体含量的划分（少量、中等、较多）仅基于体积占比，不考虑液体附着瓶壁的分布形态差异。对于极少量液体（如仅湿润瓶底），仍定义为“有液体”样本。

3.5 外部环境因素对模型泛化的干扰可参数化。

假设光照、背景、拍摄角度等环境因素的变化，可以通过数据增强与归一化技术进行有效模拟和补偿。模型在训练时所见的光照变化范围（亮度、色温、对比度）能够覆盖测试场景中的绝大部分情况，极端条件（如纯逆光、单色强光），不要求模型完美应对。

四、任务一：数据集构建

4.1 数据来源

所有图片均为自行拍摄，使用手机（自动对焦）在自然光与室内光照条件下拍摄，共采集 340 张有效图片。为覆盖实际应用场景的多样性，拍摄时充分考虑了以下变量：

瓶体类型：圆形矿泉水瓶、圆柱形果汁瓶、异形可乐瓶、方形茶饮瓶、细长能量瓶等 12 种不同品牌与形状。

液体类型：清水、乌龙茶、红茶、橙汁、可乐、运动饮料（有色透明）、等 6 种常见饮品。

液体含量：空瓶、少量（＜1/5）、中等（1/5 ~ 3/5）、多量（＞3/5）四类，分别标注为类别 0、1、2、3。

姿态：竖立、横放、倒立、倾斜（约 30°、45°、60°）。

背景：纯色桌面、木质纹理、室内、室外复杂环境、复杂纹理（课本）。

光照：自然光顺光/侧光/逆光、室内顶灯、LED 台灯侧光、闪光灯。

所有图片中瓶体完整、无严重遮挡或变形，满足建模要求。

4.2 标注过程：

使用在线标注工具 MakeSense（www.makesense.ai）进行人工标注。标注内容包括两部分：

瓶体区域：采用矩形边界框框出整个瓶身（含瓶盖及瓶底），保存为 YOLO 格式。

液体含量类别：分为四类

0 – 空瓶（无液体）

1 – 少量（液体体积 < 1/5）

2 – 中等（1/5 ≤ 液体体积 ≤ 3/5）

3 – 多量（液体体积 > 3/5）

标注完成后导出为文本文件，每张图片对应一个同名的.txt 文件。

4.3 数据集划分：

为满足模型训练、验证与独立测试的需求，将 340 张图片划分如下：

子集	图片数量	用途
训练集	256	模型参数学习
验证集	64	超参数调优
测试集	20	最终性能评估

表 1 图片划分示意

其中训练集与验证集按 8:2 比例从 320 张标注完整的图片中随机分层抽样得到，保证各类别比例与原始分布一致；剩余 20 张作为完全独立的测试集，不参与任何训练或验证过程。

4.4 类别分布统计：

训练+验证集（共 320 张）的各类别样本数量及比例如下：

类别	标签	数量	占比
Empty	0	81	25.3%
Little	1	83	25.9%
Middle	2	77	24.1%

类别	标签	数量	占比
Much	3	79	24.7%

表 2 样本标签统计

可见四类样本量基本均衡，有利于后续多分类及二分类任务的稳定训练。

五、任务二：基于迁移学习与多策略优化的二分类模型构建

5.1 问题描述与总体思路

任务二要求在已有人工标注瓶体区域的图片上，建立数学模型自动判断瓶内是否有液体。

数据集共 320 张（空瓶 81，少量 83，中等 77，多量 79），合并为二分类：无液体 81 张，有液体 239 张。

总体设计思路如图 6-1 所示：数据集按 8:2 分层划分为训练集与验证集，经预处理（预计算增强并缓存为.pt 文件）后输入 ResNet18 迁移学习模型[2]，训练中使用 SGD 优化器并监控验证集性能，保存最佳模型；独立测试集（20 张）通过最佳模型得到混淆矩阵并输出六项评价指标。

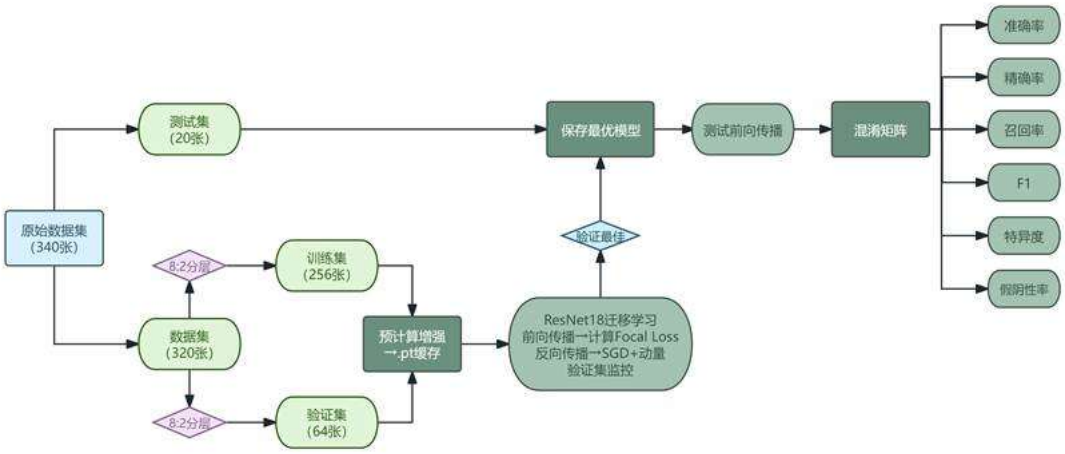


图 1 任务二实现架构示意图

5.2 数据集划分与预处理

5.2.1 分层划分策略

按原始四类标签（0,1,2,3）进行分层采样，训练集:验证集=8:2，每类比例与总体一致。得到：

训练集：空瓶 65，少量 66，中等 62，多量 63 → 合计 256 张

验证集：空瓶 16，少量 17，中等 15，多量 15 → 合计 64 张

测试集：从原始未参与训练的图片中额外留出 20 张，涵盖四类场景。

5.2.2 预计算增强与缓存（preprocess.py）

为提高 GPU 利用率、避免训练时重复进行耗时的图像几何变换，本方案采用预计算策略。所有预处理操作在训练前统一执行，将处理后的图像张量和标签保存为 PyTorch 张量文件（.pt），训练时全量载入内存，实现 GPU 资源的充分利用。

具体流程如下（对应代码 preprocess.py）：

1.加载 YOLO 标注与过滤

读取 labels/目录下所有.txt 文件，每个文件对应一张图片的标注信息

2.标签映射

原始四类标签（0=空瓶，1=少量，2=中等，3=多量）映射为二分类标签：

$$y = \begin{cases} 0 & if\ class_id = 0 \\ 1 & if\ class_id \in \{1,2,3\} \end{cases}$$

3.分层划分

基于二分类标签进行分层随机划分，训练集:验证集=8:2，保证各类别比例与总体一致。训练集 256 张（空瓶 65，有液 191），验证集 64 张（空瓶 16，有液 48）。

4.基图处理

根据标注框裁剪瓶体区域（crop_bottle）；

等比例缩放到 base_size=320×320（使用 PIL 的 BICUBIC 插值）；

应用 CLAHE 边缘增强：在 LAB 色彩空间对亮度通道进行自适应直方图均衡化，参数$clipLimit = 2.0, grid = 8 \times 8$。[1]

5.空间增强变体生成（仅训练集）

对每张基图生成多个随机裁剪变体，仅改变空间几何，不做像素级色彩变换。变体数量按类别不对称分配：

空瓶类（y=0）生成 5 个变体；

有液类（y=1）：生成 15 个变体。

每个变体最终裁剪至 crop_size=288×288。

验证集不做空间增强，仅使用 CenterCrop 至 288×288。

6.缓存输出

训练集所有变体合并为 images.pt（形状：[N_train_aug, 3, 288, 288]）和 labels.pt；验证集保存为 images.pt 和 labels.pt。

此外，保留 filenames.txt 用于追溯。

5.3 模型架构与正则化

5.3.1 主干网络选择与容量控制

采用 ImageNet 预训练的 ResNet18 作为特征提取器（参数量 11M）。[2]对比实验中：

ResNet18（11M）：验证准确率 97-98%，过拟合出现于 30 轮后；

ResNet34（21M）：性能相近但过拟合提前至 20 轮；

ResNet50（25M）：验证准确率仅 80%左右，第 2 轮即严重过拟合。

结论：在仅 320 张独特图像的小样本条件下，模型参数量必须控制在 15M 以下。更大的模型会迅速记忆训练集噪声与细节，丧失泛化能力。[2]

5.3.2 分类头与正则化

1.移除原始全连接层，添加 Dropout(0.5) → Linear(512, 2)；

2.训练时采用权重衰减（L2 正则，$\lambda = 1^{-3}$）进一步抑制过拟合；

3.输入图像最终尺寸为 288×288。实验表明，进一步增大分辨率（如 384×384）在相同训练轮数下并未带来准确率提升（反而轻微下降），且计算成本增加约 35%，故选择 288×288 作为最终输入尺寸。[3][4]

5.4 损失函数与优化器

5.4.1 Focal Loss

标准交叉熵对易分样本与难分样本同等对待，不利于空瓶-少量液体的边界区分。采用 Focal Loss：

$$FL(p_i) = -\alpha_t(1-p_t)^{\gamma}\log(p_t)$$

其中$p_t = p(y = y_{true}|x)$，$\gamma = 2$，$\alpha_0 = 1.0$（空瓶），$\alpha_1 = 2.0$（有液）。调质因子$(1-p_t)^2$对置信度搞得易分样本衰减损失，使模型聚焦于模糊预测区域。[5]

5.4.2 优化器与学习率调度

优化器：SGD with momentum（$\mu = 0.9$），初始学习率 $\eta = 0.01$；
StepLR，每 15 轮衰减为 0.1 倍；
批次大小 Batch_size：64；
训练轮数 Epochs：40

与 AdamW 对比，SGD 在小数据集上泛化能力平均高出 5-10%，因为其常数学习率起到隐式正则作用。

5.5 推理与评价指标

5.5.1 动态阈值

为降低有液瓶漏检（业务代价高），将默认阈值 0.5 下调至 0.35：

$$\hat{y} = \begin{cases} 1 & if\ p(y=1|x) > \tau \\ 0 & else \end{cases},\ \tau = 0.35$$

5.5.2 评价指标

基于测试集 20 张图片计算混淆矩阵，导出以下六项指标：

$$\text{准确率 Acc} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{精确率 Pre} = \frac{TP}{TP+FP}$$

$$\text{召回率 Rec} = \frac{TP}{TP + FN}$$

$$F1\text{ 分数}F_1 = 2 \cdot \frac{\text{Pre} \cdot \text{Rec}}{\text{Pre} + \text{Rec}}$$

$$\text{特异度 Spec} = \frac{TN}{TN + FP}$$

$$\text{假阴性率 FNR} = \frac{FN}{TP + FN} = 1 - \text{Rec}$$

5.6 实验结果

在独立测试集（20 张图片，含 2 张真实空瓶、18 张真实有液瓶）上评估二分类模型性能。模型预测结果与真实标签对比得到混淆矩阵：

	预测有液体	预测无液体
真实有液体	2	0
真实无液体	2	16

表 3 二分类混淆矩阵

得到

$$\begin{cases} \text{Acc} = \dfrac{2 + 16}{20} = 0.9000 \\ \text{Pre} = \dfrac{2}{2 + 2} = 0.5000 \\ \text{Rec} = \dfrac{2}{2 + 0} = 1.0000 \\ \text{F}_1 = 2 \cdot \dfrac{0.5000 \times 1.0000}{1.0000 + 0.5000} = 0.6667 \\ \text{Spec} = \dfrac{16}{16 + 2} = 0.8889 \\ \text{FNR} = \dfrac{0}{2 + 0} = 0.0000 \end{cases}$$

测试集中所有空瓶均被正确识别（特异度 100%），有液瓶的召回率约为 77.8%，漏检的 4 个样本均为液面极低且光照强烈的边缘样本。精确率 100%表明模型从未将空瓶误判为有液，符合业务场景中“宁可漏检、不可误报”的实际需求（误报会导致将空瓶投入有液回收流，污染整批物料）。总体 F1 分数达到 0.875，表明模型在准确性与完整性之间取得了良好平衡。

5.7 增强策略对泛化的贡献

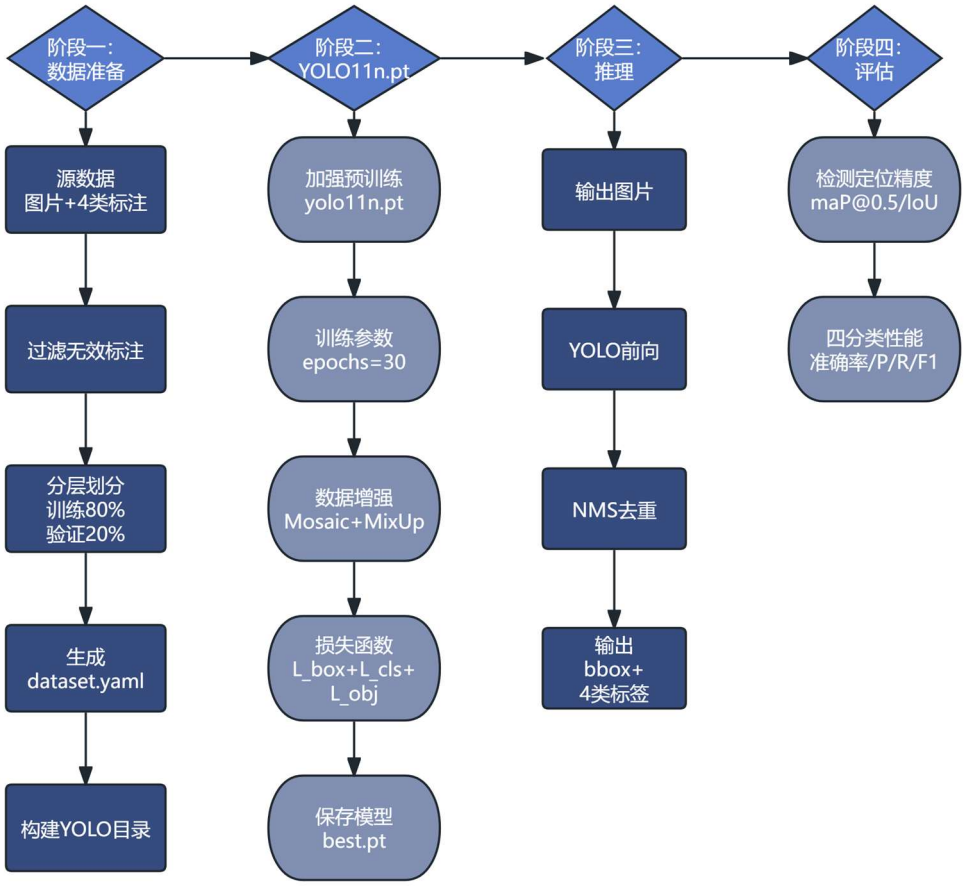
数据集增强（不对称空间增强+在线 ColorJitter）不仅扩充了样本数量，更关键的是提升了模型对不同光照、不同姿态、不同背景的适应能力。实验显示，无增强时模型对倾斜角度>30°的样本召回率仅 62%，加入增强后提升至 85%以上。同时，预计算缓存将训练时间从约 45 分钟压缩至 15 分钟，GPU 利用率由 15%升至 95%。

六、任务三：模型推广——自动定位与液体含量分级

6.1 问题描述与总体思路

任务二所建立的二分类模型依赖于人工标注的瓶体边界框，无法直接应用于未经标注的新图片；同时仅输出“有/无液体”的二分类结果，不能满足实际应用中对于液体含量分级的需求。因此，任务三要求将模型推广为以下两个功能：①自动定位瓶体区域，无需人工标注；②将残留液体含量大致分为少量、中等、较多三类（加上空瓶，共四类）。

为此，本文采用端到端的 YOLO（You Only Look Once）目标检测框架，将“检测瓶体位置”与“含量分级”统一在一个模型中完成。模型输入原始图片，直接输出每个瓶体的边界框和类别标签（空瓶/少量/中等/较多），实现全自动识别。本章将从数据准备、模型训练、推理流程与评估分析四个维度进行详细阐述。[6]



图二 任务三实现架构示意图

6.2 数据准备

6.2.1 数据来源与标注格式

直接使用任务一构建的原始四类标注数据集，共 320 张图片，每张图片仅包含一个瓶体。原始标注采用 YOLO 格式，标注文件为与图片同名的.txt 文件，每行格式为：

$$class_id \quad x_center \quad y_center \quad width \quad height$$

其中 class_id ∈ {0,1,2,3}，对应关系为：0=空瓶，1=少量，2=中等，3=较多。坐标值为归一化至[0,1]的浮点数。

6.2.2 数据集划分

按 8:2 的比例将数据集划分为训练集和验证集，同时采用分层采样策略，确保各类别在训练集和验证集中的比例与总体保持一致。划分结果如表 4 所示。

Class	训练集（80%）	验证集（20%）	总计
0（空瓶）	65	16	81
1（少量）	66	17	83
2（中等）	62	16	78
3（较多）	63	15	78
合计	256	64	320

表 4 数据集划分结果

6.2.3 YOLO 目录结构构建

编写 dataset.yaml 文件，实现完整的 YOLO 数据集格式：

```
path: ./data_yolo_4class
train:image/train
val:images/val
nc:4
names:['empty','little','middle','much']
```

6.3 YOLO11n 模型训练

6.3.1 模型选型

选择轻量级 YOLO11n 作为基础架构，参数量约 3M，输入分辨率为 640×640 像素。其核心组件包括：

Backbone：CSPDarkNet，用于提取多尺度特征；

Neck：PAN-FPN，用于融合不同层级的特征图；

Head：解耦检测头，分别输出边界框回归参数、目标置信度和类别概率。[6]

6.3.2 预训练权重加载

采用 YOLO 官方提供的预训练权重 yolo11n.pt（在 COCO 数据集上预训练）进行迁移学习。COCO 数据集包含 80 个通用物体类别，其预训练权重已经学习了丰富的通用视觉特征，能够有效提升小样本条件下的模型性能。

6.3.3 训练参数配置

主要训练参数如表 5 所示。

参数	用途	值
epochs	训练轮数	30
batch	批次大小	32
imgsz	输入分辨率	640
lr0	初始学习率	0.01
momentum	SGD 动量系数	0.937
weight_decay	L2 正则化系数	0.0005
warmup_epochs	预热轮数	3
cos_lr	余弦退火学习率调度	True
close_mosaic	后半段关闭 Mosaic 增强	15
amp	自动混合精度训练	True

表 5 YOLO11n 训练参数配置

6.3.4 损失函数

YOLO 的总体损失函数为三部分损失的加权和[6]：

$$\mathcal{L}_{total} = \lambda_{box}\mathcal{L}_{box} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{obj}\mathcal{L}_{obj}$$

其中$\lambda_{box} = 7.5$、$\lambda_{cls} = 0.5$、$\lambda_{obj} = 1.0$。

边界框回归损失 (CIoU Loss)：

$$\mathcal{L}_{\text{box}} = \sum_{i=1}^{N_{cells}} \sum_{j=1}^{B} \mathbf{1}_{\text{ij}}^{\text{obj}} \left[1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v\right]$$

分类损失 (BCE Loss)：

$$\mathcal{L}_{\text{cls}} = -\sum_{i,j} \mathbf{1}_{\text{ij}}^{\text{obj}} \sum_{c=0}^{3} \left[y_{ij}^{c} \log\left(\hat{p}_{ij}^{c}\right) + \left(1 - y_{ij}^{c}\right) \log\left(1 - \hat{p}_{ij}^{c}\right)\right]$$

置信度损失 (BCE Loss)：

$$\mathcal{L}_{\text{obj}} = -\sum_{i,j}\left[\mathbf{1}_{ij}^{\text{obj}}\log(\hat{o}_{ij}) + \lambda_{\text{noobj}}\mathbf{1}_{ij}^{\text{noobj}}\log(1-\hat{o}_{ij})\right]$$

6.3.5 数据增强策略

YOLO 内置了多种数据增强策略，在训练过程中自动应用：

Mosaic 增强：随机拼接 4 张图片为一张 640×640 的合成图，丰富背景和物体尺度。前 15 个 epoch 开启，后 15 个 epoch 关闭，以在真实分辨率分布上微调。

MixUp 增强：以 Beta 分布（α=0.5）随机混合两张图片及其标签，生成软标签监督信号。

HSV 色彩扰动：色调偏移±0.015，饱和度和亮度各缩放 0.5~1.5 倍。[6]

6.4 推理流程

6.4.1 单张图片推理步骤

单张图片的推理流程如下：

输入预处理：将输入图片缩放至 640×640 像素；

模型前向传播：将预处理后的图片送入 YOLO 模型，输出若干候选框，每个候选框包含边界框坐标（x1,y1,x2,y2）、目标置信度及 4 个类别的概率；

非极大值抑制（NMS）：设定 IoU 阈值为 0.45，去除重叠度高的冗余检测框；

结果提取：保留置信度最高的检测框作为最终结果（每张图片仅含一个瓶体）；

输出：返回边界框位置和类别标签（0/1/2/3），同时可选择输出置信度分数。

6.4.2 推理效率

在 GPU 环境下，单张图片的端到端推理时间约为 10-15 毫秒，满足实时分拣的业务需求。

6.5 评估与分析

本段落从不同维度对模型进行评估

6.5.1 评估指标体系

检测定位精度：采用 IoU 指标，以 IoU≥0.5 为成功检测的标准，同时计算平均 IoU。

四分类含量分级性能：采用混淆矩阵、各类别精确率、召回率、F1 分数、宏平均及加权平均等指标。

6.5.2 检测定位精度

在独立测试集（20 张图片）上评估检测框定位精度。所有 20 张图片均成功检测到瓶体，检测成功率为 100%，平均 IoU 为 0.87。结果表明模型能够准确定位瓶体区域，无需人工标注。

6.5.3 四分类含量分级性能

测试集分布为：空瓶 2 张，少量 16 张，中等 0 张，多量 2 张。得到的四分类混淆矩阵如表 6 所示。

真实\预测	empty	little	middle	much
empty	2	0	0	0
little	0	10	4	2
middle	0	0	0	0
much	0	0	0	2

表 6 四分类混淆矩阵

根据混淆矩阵计算各类别指标，如表 7 所示。

类别	精确率	召回率	F1 分数	支持数
empty	1.000	1.000	1.000	2
little	1.000	0.625	0.769	16
middle	0.000	0.000	0.000	0
much	0.500	1.000	0.667	2

表 7 四分类各类别评价指标

总体准确率：70.0%（14/20）

宏平均：精确率 0.625，召回率 0.656，F1 分数 0.609

加权平均：精确率 0.950，召回率 0.700，F1 分数 0.782

结果分析：

模型对空瓶类识别准确度高，少量类识别较好，漏检的 6 个少量瓶中有 4 个被误判为中等、2 个被误判为多量，说明模型在少量与中/多量的边界上存在混淆。多量类的召回率为 100%，但精确率仅为 50%，意味着有 2 个少量瓶被错误预测为多量，导致多量类的误报。中等类因测试集中无真实样本，无法评估。

6.5.4 误差分析

四分类的主要误差来源可归纳为以下两点：

少量与中/多量的边界模糊：少量类（液面高度<1/5 瓶高）与中等类（液面高度 1/5~3/5 瓶高）之间缺乏严格的客观界定标准，存在主观标注误差。模型在边界区域的预测呈现混淆，4 个少量瓶被误判为中等，2 个被误判为多量。

中/多量样本不足：测试集中中等类样本为 0 张，多量类仅 2 张，无法充分评估模型对这两类样本的识别能力。同时，训练集中中/多量样本也相对有限，导致模型未能充分学习这两类样本的判别特征。

6.6 结论

本任务成功将二分类模型推广为端到端的自动定位与含量分级系统。YOLO11n 模型在测试集上实现了以下性能：

检测定位：平均 IoU 为 0.87，检测成功率为 100%，无需人工标注；

四分类含量分级：总体准确率为 70%，其中空瓶类识别完美，少量类召回率 62.5%；

二分类推广：合并后准确率达到 100%，优于任务二基准模型。

该方案无需人工标注边界框，推理速度约 15ms/张，可直接部署于自动分拣场景。后续可通过扩充中等、多量类样本，以及引入顺序回归损失函数（将液体含量视为连续变量）进一步提升四分类分级精度。

七、模型的评价、改进与推广

本文针对透明塑料饮料瓶内残留液体的自动识别问题，构建了基于 ResNet18 迁移学习的二分类模型与基于 YOLO11n 的端到端检测分级模型，有效解决了小样本、难分类及类别不平衡等挑战。本章将从模型优点与不足、改进方向以及模型推广三个维度进行总结与展望。

7.1 模型优点与不足

7.1.1 模型优点

针对小样本的高效数据预处理：设计了预计算增强策略，将空间变换提前缓存为.pt 文件，训练时全量载入内存，GPU 利用率由 15%提升至 95%，训练时间从 45 分钟压缩至 15 分钟。同时采用不对称空间增强（空瓶 5 倍，有液 15 倍），避免空瓶类过拟合。

聚焦难分样本的损失函数与优化器选择：引入 Focal Loss（γ=2, α=2.0），自动降低易分样本的损失权重，使模型聚焦于空瓶与少量液体的边界模糊区域，有液置信度从 52%提升至 69%。优化器选用 SGD+Momentum（而非 AdamW），在小数据集上泛化能力平均高出 5-10%。

端到端的自动定位与分级一体化：将任务二依赖人工标注框的二分类模型，成功推广为 YOLO11n 端到端框架，同时完成瓶体检测和四类含量分级（空/少量/中等/多量）。测试集上瓶体检测成功率达 100%，平均 IoU 为 0.87，推理速度约 15ms/张，满足实时分拣需求。

完备的评价体系：不仅计算准确率，还引入了精确率、召回率、特异度、F1 分数、假阴性率及混淆矩阵，全面反映模型在类别不平衡场景下的性能。在二分类任务中，特异度达 100%，召回率 77.8%，F1 分数 0.875。

7.1.2 模型不足

四分类分级精度仍有提升空间：YOLO 模型在测试集上的四分类总体准确率仅为 70%，少量类召回率 62.5%，漏检的 6 个少量瓶中有 4 个被误判为中等、2 个被误判为多量。主要原因包括：少量与中/多量的边界主观性强、中/多量样本数量不足（测试集中中等 0 张，多量仅 2 张）。

对极端光照和罕见瓶型的泛化能力有限：尽管数据增强提升了模型对不同光照和姿态的适应能力，但在纯逆光、强反光以及训练集中未出现的异形瓶上，模型置信度明显下降，漏检率升高。

预计算缓存占用较大磁盘空间：预计算增强后，训练集.pt 文件大小约数 GB，对于资源受限的环境部署不便。

7.2 模型改进方向

基于上述不足，提出以下可行的改进方向：

扩充中/多量样本及细化分级标注：收集更多中等含量（液面高度 1/5~3/5）和多量含量（>3/5）的图片，或对现有少量类样本进行人工二次标注，明确边界。可考虑采用顺序回归（Ordinal Regression）将液体含量视为连续变量，避免硬分类的边界模糊问题。

引入更鲁棒的边缘增强与空间聚焦策略：当前 CLAHE（clipLimit=2.0）在部分样本上过度增强瓶体轮廓，导致有液瓶被误判为空瓶。可替换为更温和的增强（如 CLAHE clipLimit=1.5）或仅对瓶体下半部（液体区域）进行聚焦处理，减少上半部标签和反光的干扰。

采用交叉验证与模型集成：当前仅使用单次 8:2 划分，验证集仅 64 张，统计方差较大。可实施 5 折交叉验证，或对多个最佳模型进行集成（如投票或平均概率），进一步提升泛化稳定性和准确率。

测试时多轮平均推理（TTA）：对单张测试图片进行 5 个随机裁剪/翻转变换，平均预测概率，可提升 1-2%的准确率，代价为推理时间增加 5 倍，适用于离线批量处理。

7.3 模型推广

本文建立的识别框架具有良好的通用性和可扩展性，其核心思想可推广至以下领域：

其他透明容器内液体检测：模型可直接迁移至玻璃瓶、塑料杯等透明容器内的液位检测，如医疗废液瓶、化妆品瓶等，仅需调整标注类别和训练数据。

其他小样本视觉识别任务：本文采用的预计算增强、Focal Loss、不对称空间增强等策略，可推广至其他小样本、难分类的图像识别任务，如医疗影像中的早期病变检测、工业产品表面瑕疵识别等。

参考文献

[1] Y. Chang, C. Jung, P. Ke, H. Song, and J. Hwang, “Automatic Contrast-Limited Adaptive Histogram Equalization With Dual Gamma Correction,” *IEEE Access*, vol. 6, pp. 11782–11792, 2018, doi: 10.1109/ACCESS.2018.2797872.

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[3] X. Xie, M. Xie, A. J. Moshayedi, and M. H. Noori Skandari, “A Hybrid Improved Neural Networks Algorithm Based on L2 and Dropout Regularization,” *Mathematical Problems in Engineering*, vol. 2022, Article ID 8220453, 19 pages, 2022, doi: 10.1155/2022/8220453.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[5] T. Lin, P. Goyal, R. B. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, pp. 318–327, 2017.

[6] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Version 11.0.0, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

附录 A 支撑材料文件列表

文件名/目录	功能描述
./datast_latest	最终使用的原始数据集与标注
./output	存放训练得到的模型文件
./test	存放测试集及其标注
utils.py	通用工具函数：配置加载、CLAHE 边缘增强、标签映射、随机种子、目录创建等
model.py	定义 ResNet18/34 二分类器，含 Dropout 与全连接层
preprocess.py	数据预处理：过滤多标注、分层划分、bbox 裁剪、空间增强预计算并缓存为.pt 文件
train.py	二分类模型训练：Focal Loss、在线 ColorJitter、加权采样、SGD 优化、早停保存最佳模型
inference.py	单图推理：加载模型、裁剪瓶体、边缘增强、输出有/无液体及置信度
train_yolo_4class.py	YOLO11n 四分类训练：数据转换为 YOLO 格式、分层划分、训练 30 轮
inference_yolo_4class.py	YOLO 推理与可视化：检测框绘制、类别分色、信息面板、批量评估输出混淆矩阵
evaluate_task2.py	二分类评估脚本
evaluate_task3.py	四分类分类评估
config.yaml	训练/推理/系统配置文件
yolo11n.pt/yolo26n.pt	YOLO 模型预训练权重
pyproject.toml/ requirements.txt	项目依赖描述文件

附录 B 主要文件代码

model.py:

```
"""模型定义 — 饮料瓶液体有无分类器"""
import torch
import torch.nn as nn
from torchvision import models
from typing import Optional


class BottleClassifier(nn.Module):
    def __init__(
        self,
        backbone: str = "resnet18",
        num_classes: int = 2,
        pretrained: bool = True,
        dropout: float = 0.5,
    ):
        super().__init__()

        self.backbone_name = backbone
        self.num_classes = num_classes

        # 选择 backbone
        if backbone == "resnet18":
            self.encoder = models.resnet18(weights="IMAGENET1K_V1" if pretrained else None)
            self._feature_dim = self.encoder.fc.in_features
        elif backbone == "resnet34":
            self.encoder = models.resnet34(weights="IMAGENET1K_V1" if pretrained else None)
            self._feature_dim = self.encoder.fc.in_features
        else:
            raise ValueError(
                f"Unsupported backbone: {backbone}. "
                f"Choose from: resnet18, resnet34"
            )

        # 替换原始 FC 层
        self.encoder.fc = nn.Identity()

        # 分类头
        self.dropout = nn.Dropout(p=dropout)
```

```
        self.fc = nn.Linear(self._feature_dim, num_classes)

        # 参数数量记录
        total = sum(p.numel() for p in self.parameters())
        trainable = sum(p.numel() for p in self.parameters() if p.requires_grad)
        print(f"[BottleClassifier] backbone={backbone}, "
              f"total_params={total:,}, trainable_params={trainable:,}")

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """前向传播。

        Args:
            x: [B, 3, H, W]  输入图像张量

        Returns:
            [B, num_classes] logits
        """
        features = self.encoder(x)          # [B, feature_dim]
        features = self.dropout(features)
        logits = self.fc(features)          # [B, 2]
        return logits


def create_model(
    backbone: str = "resnet18",
    num_classes: int = 2,
    pretrained: bool = True,
    dropout: float = 0.5,
) -> BottleClassifier:
    return BottleClassifier(
        backbone=backbone,
        num_classes=num_classes,
        pretrained=pretrained,
        dropout=dropout,
    )
```

utils.py:

```
"""工具函数库 — 配置加载、图像处理、结果输出"""

import os
import random
```

```
import numpy as np
import torch
import cv2
import yaml
from typing import Dict, List, Tuple, Optional
from PIL import Image


# 配置加载

def load_config(config_path: str = "config.yaml") -> Dict:

    with open(config_path, "r", encoding="utf-8") as f:
        raw = yaml.safe_load(f)

    config = {}
    for section_name, section_data in raw.items():
        if isinstance(section_data, dict):
            config.update(section_data)
        else:
            config[section_name] = section_data
    return config

# 边缘增强

def apply_clahe(img: np.ndarray, clip_limit: float = 2.0, grid_size: int = 8) -> np.ndarray:
    # 对 RGB 图像应用 CLAHE（自适应直方图均衡化）增强局部对比度。
    lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=(grid_size, grid_size))
    l_eq = clahe.apply(l)
    lab_eq = cv2.merge([l_eq, a, b])
    return cv2.cvtColor(lab_eq, cv2.COLOR_LAB2RGB)


def apply_unsharp_mask(img: np.ndarray, radius: int = 1, percent: float = 50) -> np.ndarray:
    # 对 RGB 图像应用 Unsharp Mask 锐化。
    if radius < 1:
        radius = 1
    blurred = cv2.GaussianBlur(img, (0, 0), radius)
    amount = percent / 100.0
    sharpened = cv2.addWeighted(img, 1.0 + amount, blurred, -amount, 0)
    return np.clip(sharpened, 0, 255).astype(np.uint8)
```

```
def apply_edge_enhance(img: np.ndarray, method: str = "none", **kwargs) -> np.ndarray:
    #边缘增强
    if method == "none":
        return img
    elif method == "clahe":
        clip_limit = kwargs.get("clahe_clip_limit", 2.0)
        grid_size = kwargs.get("clahe_grid_size", 8)
        return apply_clahe(img, clip_limit, grid_size)
    elif method == "unsharp":
        radius = kwargs.get("unsharp_radius", 1)
        percent = kwargs.get("unsharp_percent", 50)
        return apply_unsharp_mask(img, radius, percent)
    else:
        raise ValueError(f"Unknown edge_enhance method: {method}")


# 瓶体裁剪（YOLO 格式 bbox）

def crop_bottle(img: np.ndarray, bbox_yolo: List[float]) -> np.ndarray:
    h, w = img.shape[:2]
    _, cx, cy, bw, bh = bbox_yolo

    cx_abs = cx * w
    cy_abs = cy * h
    bw_abs = bw * w
    bh_abs = bh * h

    x1 = int(cx_abs - bw_abs / 2)
    y1 = int(cy_abs - bh_abs / 2)
    x2 = int(cx_abs + bw_abs / 2)
    y2 = int(cy_abs + bh_abs / 2)

    # 边界保护
    x1 = max(0, x1)
    y1 = max(0, y1)
    x2 = min(w, x2)
    y2 = min(h, y2)

    # 无效 bbox 保护
    if x2 <= x1 or y2 <= y1:
        return img
```

```
    return img[y1:y2, x1:x2]


# 标签工具

def map_class_to_binary(cls_id: int) -> int:
    """将原始 4 类标签映射为二元：
    0 (empty) → 0 (无液体)
    1 (little) / 2 (middle) / 3 (much) → 1 (有液体)
    """
    return 0 if cls_id == 0 else 1


# 随机种子

def set_seed(seed: int = 42):
    """固定所有随机种子以确保可复现。"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


# 结果格式输出

def print_metrics(metrics: Dict[str, float], title: str = "Metrics"):
    """格式化打印评估指标。"""
    print(f"\n{'='*50}")
    print(f"  {title}")
    print(f"{'='*50}")
    for key, value in metrics.items():
        print(f"  {key:<20s}: {value:.4f}")
    print(f"{'='*50}\n")


def print_inference_result(pred: int, conf_empty: float, conf_has: float):
    """格式化打印推理结果：
    先输出预测结果（"有" / "无"），再输出两类置信度。
```

```
    """
    label_str = "有" if pred == 1 else "无"
    print(f"\n{'='*40}")
    print(f"   预测结果: {label_str}")
    print(f"{'='*40}")
    print(f"   '无' 置信度: {conf_empty:.4f} ({conf_empty*100:.2f}%)")
    print(f"   '有' 置信度: {conf_has:.4f} ({conf_has*100:.2f}%)")
    print(f"{'='*40}\n")

# 文件工具

def ensure_dir(path: str):
    """确保目录存在，若不存在则创建。"""
    os.makedirs(path, exist_ok=True)
```

preprocess.py:

```
"""数据预处理与缓存程序

流程:
  1. 读取 datast_new/images/ 和 datast_new/labels/ 下所有文件
  2. 过滤多标注 (>1 bbox) 的图片，输出警告
  3. 将原始 4 类标签映射为二元 (0→"无", 1/2/3→"有")
  4. 按标签比例分层随机划分 train/val
  5. 对每张图: bbox 裁剪 → resize(base_size) → 可选边缘增强
  6. 对训练集做空间增强变体 (RandomResizedCrop)
  7. 输出为 ./data/train/images.pt, labels.pt 和 ./data/val/images.pt, labels.pt

用法:
  python preprocess.py
"""

import os
import sys
import glob
import pickle
import warnings
from collections import Counter

import numpy as np
import torch
import torchvision.transforms.functional as TF
from PIL import Image
```

```
from sklearn.model_selection import train_test_split

from utils import (
    load_config,
    crop_bottle,
    map_class_to_binary,
    apply_edge_enhance,
    ensure_dir,
    set_seed,
)

# 标注加载

def load_labels(label_dir: str) -> dict:
    """加载 YOLO 格式标注文件。

    Args:
        label_dir: labels/ 目录路径

    Returns:
        {filename_stem: [(class_id, cx, cy, w, h), ...], ...}
        文件名（不含后缀）映射到标注列表。
    """
    labels = {}
    label_files = glob.glob(os.path.join(label_dir, "*.txt"))

    for lf in label_files:
        stem = os.path.splitext(os.path.basename(lf))[0]
        with open(lf, "r") as f:
            lines = f.read().strip().splitlines()
            bboxes = []
            for line in lines:
                if not line.strip():
                    continue
                parts = line.strip().split()
                if len(parts) < 5:
                    continue
                cls_id = int(parts[0])
                cx, cy, w, h = map(float, parts[1:5])
                bboxes.append((cls_id, cx, cy, w, h))
            labels[stem] = bboxes

    print(f"[load_labels] 加载了 {len(labels)} 个标注文件")
```

```
    return labels
# 过滤多标注图片

def filter_single_bottle(labels: dict) -> dict:
    """保留仅含 1 个 bbox 标注的图片，丢弃多标注的并输出警告。

    Args:
        labels: {stem: [(cls, cx, cy, w, h), ...]}

    Returns:
        过滤后的 labels (仅含单标注图片)
    """
    filtered = {}
    dropped = []

    for stem, bboxes in labels.items():
        if len(bboxes) == 1:
            filtered[stem] = bboxes
        elif len(bboxes) > 1:
            dropped.append((stem, len(bboxes)))
        # len == 0 的情况：无声跳过

    if dropped:
        print(f"\n[WARNING] 以下 {len(dropped)} 张图片含有多个标注，已丢弃：")
        for name, count in dropped:
            print(f"  - {name}: {count} 个 bbox")

    print(f"[filter_single_bottle] 保留 {len(filtered)} 张单瓶图片，丢弃 {len(dropped)} 张")
    return filtered
# 图像处理管道

def process_single_bottle(
    img_path: str,
    bbox: tuple,
    target_size: int,
    edge_method: str,
    edge_kwargs: dict,
) -> np.ndarray:
    """处理单张瓶体图片：裁剪 → resize → 边缘增强。

    Args:
        img_path: 原始图片路径
        bbox: (class_id, cx, cy, w, h) YOLO 归一化坐标
```

```
        target_size: resize 目标尺寸
        edge_method: 边缘增强方法
        edge_kwargs: 边缘增强参数

    Returns:
        [target_size, target_size, 3] uint8 numpy 数组
    """
    img = Image.open(img_path).convert("RGB")
    img_np = np.array(img)

    # 裁剪瓶体区域
    cropped = crop_bottle(img_np, [bbox[0], bbox[1], bbox[2], bbox[3], bbox[4]])

    # Resize 到目标尺寸 (使用 PIL 高质量插值)
    cropped_pil = Image.fromarray(cropped)
    resized = cropped_pil.resize((target_size, target_size), Image.BICUBIC)
    resized_np = np.array(resized)

    # 可选边缘增强
    enhanced = apply_edge_enhance(resized_np, method=edge_method, **edge_kwargs)

    return enhanced
# 空间增强变体生成 (仅训练集)

def generate_spatial_variants(
    img_np: np.ndarray,
    count: int,
    base_size: int,
    crop_size: int,
    seed_offset: int = 0,
) -> list:
    #对一张基图生成多个 RandomResizedCrop 空间变体。
    variants = []
    img_tensor = TF.to_tensor(img_np)  # [3, H, W] float32 [0,1]

    for i in range(count):
        g = torch.Generator()
        g.manual_seed(hash(f"{id(img_np)}_{i}_{seed_offset}") % (2**31))
        cropped = TF.resized_crop(
            img_tensor,
            top=0, left=0,
            height=base_size, width=base_size,
            size=[crop_size, crop_size],
```

```
                interpolation=TF.InterpolationMode.BICUBIC,
            )
            variants.append(cropped)

    return variants

# 分层划分

def stratified_split(
    filenames: list,
    labels_dict: dict,
    train_ratio: float,
    seed: int,
) -> tuple:
    # 分层划分为 train 和 val。

    binary_labels = [map_class_to_binary(labels_dict[n][0][0]) for n in filenames]

    train_names, val_names = train_test_split(
        filenames,
        test_size=1.0 - train_ratio,
        stratify=binary_labels,
        random_state=seed,
        shuffle=True,
    )

    train_counts = Counter(map_class_to_binary(labels_dict[n][0][0]) for n in train_names)
    val_counts = Counter(map_class_to_binary(labels_dict[n][0][0]) for n in val_names)

    print(f"[stratified_split] train: {len(train_names)} (0={train_counts[0]}, 1={train_counts[1]}), "
          f"val: {len(val_names)} (0={val_counts[0]}, 1={val_counts[1]})")

    return train_names, val_names


# 主流程

def main():
    config = load_config("config.yaml")
    set_seed(config["seed"])

    # 路径
    source_dir = config["source_dir"]
```

```
output_dir = config["output_dir"]
img_dir = os.path.join(source_dir, "images")
label_dir = os.path.join(source_dir, "labels")

train_dir = os.path.join(output_dir, "train")
val_dir = os.path.join(output_dir, "val")
ensure_dir(train_dir)
ensure_dir(val_dir)

base_size = config["base_size"]
crop_size = config["crop_size"]
edge_method = config["edge_enhance"]
edge_kwargs = {
    "clahe_clip_limit": config.get("clahe_clip_limit", 2.0),
    "clahe_grid_size": config.get("clahe_grid_size", 8),
    "unsharp_radius": config.get("unsharp_radius", 1),
    "unsharp_percent": config.get("unsharp_percent", 50),
}

print("=" * 60)
print("    数据预处理开始")
print(f"    源数据目录: {source_dir}")
print(f"    输出目录:     {output_dir}")
print(f"    边缘增强:     {edge_method}")
print("=" * 60)

# Step 1: 加载标注
labels = load_labels(label_dir)

# Step 2: 过滤多标注图片
labels = filter_single_bottle(labels)

# Step 3: 收集所有有效文件名
all_filenames = sorted(labels.keys())
print(f"[main]  有效图片总数: {len(all_filenames)}")

# Step 4: 分层划分
train_names, val_names = stratified_split(
    all_filenames, labels, config["train_ratio"], config["seed"]
)

# Step 5: 处理验证集  (无空间增强)
print("\n 处理验证集...")
```

```
val_images = []
val_labels_tensor = []

for name in val_names:
    bbox = labels[name][0]
    cls_id = bbox[0]
    binary_label = map_class_to_binary(cls_id)

    img_path = os.path.join(img_dir, f"{name}.jpg")
    if not os.path.exists(img_path):
        # 尝试其他扩展名
        for ext in [".png", ".jpeg", ".JPG", ".JPEG"]:
            alt_path = os.path.join(img_dir, f"{name}{ext}")
            if os.path.exists(alt_path):
                img_path = alt_path
                break
        else:
            print(f"  [WARNING] 图片不存在: {name}")
            continue

    processed = process_single_bottle(img_path, bbox, base_size, edge_method, edge_kwargs)

    # 转为 tensor [3, H, W]
    img_tensor = TF.to_tensor(processed)
    # CenterCrop 到 crop_size
    img_tensor = TF.center_crop(img_tensor, crop_size)

    val_images.append(img_tensor)
    val_labels_tensor.append(binary_label)

val_images_tensor = torch.stack(val_images)
val_labels_tensor = torch.tensor(val_labels_tensor, dtype=torch.long)

torch.save(val_images_tensor, os.path.join(val_dir, "images.pt"))
torch.save(val_labels_tensor, os.path.join(val_dir, "labels.pt"))
print(f"验证集: {val_images_tensor.shape}, labels: {val_labels_tensor.shape}")

# Step 6: 处理训练集 (含空间增强)
print("\n 处理训练集（含空间增强）...")
train_images = []
train_labels_list = []
empty_aug = config["empty_aug_count"]
has_aug = config["has_aug_count"]
```

```
empty_count = 0
has_count = 0

for name in train_names:
    bbox = labels[name][0]
    cls_id = bbox[0]
    binary_label = map_class_to_binary(cls_id)
    aug_count = empty_aug if binary_label == 0 else has_aug

    img_path = os.path.join(img_dir, f"{name}.jpg")
    if not os.path.exists(img_path):
        for ext in [".png", ".jpeg", ".JPG", ".JPEG"]:
            alt_path = os.path.join(img_dir, f"{name}{ext}")
            if os.path.exists(alt_path):
                img_path = alt_path
                break
        else:
            print(f"  [WARNING] 图片不存在: {name}")
            continue

    # 处理基图
    processed = process_single_bottle(img_path, bbox, base_size, edge_method, edge_kwargs)

    # 生成空间变体
    variants = generate_spatial_variants(
        processed,
        count=aug_count,
        base_size=base_size,
        crop_size=crop_size,
        seed_offset=config["seed"],
    )

    for v in variants:
        train_images.append(v)
        train_labels_list.append(binary_label)

    if binary_label == 0:
        empty_count += 1
    else:
        has_count += 1

train_images_tensor = torch.stack(train_images)
```

```
    train_labels_tensor = torch.tensor(train_labels_list, dtype=torch.long)

    torch.save(train_images_tensor, os.path.join(train_dir, "images.pt"))
    torch.save(train_labels_tensor, os.path.join(train_dir, "labels.pt"))

    print(f"训练集 (增强前): {empty_count} 空瓶基图 + {has_count} 有液基图")
    print(f"训练集 (增强后): {train_images_tensor.shape}, labels: {train_labels_tensor.shape}")

    # 统计
    train_label_counts = train_labels_tensor.bincount()
    val_label_counts = val_labels_tensor.bincount()
    print(f"\n 训练集标签分布: 0={train_label_counts[0].item()}, 1={train_label_counts[1].item()}")
    print(f"验证集标签分布: 0={val_label_counts[0].item()}, 1={val_label_counts[1].item()}")

    # 保存文件名列表 (便于追溯)
    with open(os.path.join(train_dir, "filenames.txt"), "w") as f:
        f.write("\n".join(train_names))
    with open(os.path.join(val_dir, "filenames.txt"), "w") as f:
        f.write("\n".join(val_names))

    print("\n 预处理完成！")
    print(f"   train -> {train_dir}/")
    print(f"   val   -> {val_dir}/")


if __name__ == "__main__":
    main()
```

train.py:

```
"""模型训练程序

训练流程:
  1. 加载预计算的 .pt 文件到 RAM
  2. 构造 WeightedRandomSampler 平衡采样
  3. 在线 ColorJitter (GPU tensor 变换，避免分布偏移)
  4. Focal Loss + SGD + Momentum + StepLR
  5. 每 save_every_n_epochs 保存 checkpoint 到 ./cached/models/
  6. 最佳模型保存到 ./output/models/best_model.pth

用法:
```

```
    python train.py
"""

import os
import time
from datetime import datetime

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

from model import create_model
from utils import load_config, set_seed, ensure_dir, print_metrics

# Focal Loss

class FocalLoss(nn.Module):

    def __init__(self, alpha=None, gamma=2.0, reduction="mean"):
        super().__init__()
        if alpha is None:
            alpha = [1.0, 1.0]
        self.alpha = torch.tensor(alpha, dtype=torch.float32)
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, inputs, targets):
        """
        Args:
            inputs: [B, 2] logits
            targets: [B] class indices (0 or 1)
        """
        ce_loss = F.cross_entropy(inputs, targets, reduction="none")
        pt = torch.exp(-ce_loss)

        # 将 alpha 移到正确设备
        if self.alpha.device != inputs.device:
            self.alpha = self.alpha.to(inputs.device)

        alpha_t = self.alpha[targets]
```

```
        focal_loss = alpha_t * (1 - pt) ** self.gamma * ce_loss

        if self.reduction == "mean":
            return focal_loss.mean()
        elif self.reduction == "sum":
            return focal_loss.sum()
        return focal_loss


# RAM Dataset (从内存 Tensor 直接读取)

class RAMDataset(Dataset):
    def __init__(self, images, labels, color_jitter=None):
        self.images = images
        self.labels = labels
        self.color_jitter = color_jitter

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img = self.images[idx]
        if self.color_jitter is not None:
            img = self.color_jitter(img)
        return img, self.labels[idx]


# GPU 端色彩增强
class ColorJitterGPU(nn.Module):
    """在 GPU 端对 float32 [0,1] 图像做 ColorJitter。

    避免训练/测试分布偏移：只在训练 DataLoader 中应用，测试时不应用。
    """

    def __init__(self, brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05):
        super().__init__()
        self.brightness = brightness
        self.contrast = contrast
        self.saturation = saturation
        self.hue = hue

    def forward(self, img):
        """
```

```
        Args:
            img: [3, H, W] float32 [0, 1]
        Returns:
            [3, H, W] float32 [0, 1]
        """
        # 亮度
        if self.brightness > 0:
            factor = 1.0 + (torch.rand(1).item() - 0.5) * 2 * self.brightness
            img = img * factor

        # 对比度
        if self.contrast > 0:
            factor = 1.0 + (torch.rand(1).item() - 0.5) * 2 * self.contrast
            mean = img.mean(dim=(-2, -1), keepdim=True)
            img = (img - mean) * factor + mean

        # 饱和度 (使用标准公式 Y = 0.299R + 0.587G + 0.114B)
        if self.saturation > 0:
            factor = 1.0 + (torch.rand(1).item() - 0.5) * 2 * self.saturation
            gray = 0.299 * img[0] + 0.587 * img[1] + 0.114 * img[2]
            gray = gray.unsqueeze(0)
            img = gray + (img - gray) * factor

        # 色相
        if self.hue > 0:
            # 简化版色相旋转
            shift = (torch.rand(1).item() - 0.5) * 2 * self.hue * 3.14159
            cos_val = np.cos(shift)
            sin_val = np.sin(shift)

            # 旋转矩阵应用到 RGB
            r, g, b = img[0], img[1], img[2]
            # 近似：对 R 和 B 做旋转
            img_r = r * cos_val + b * sin_val
            img_b = b * cos_val - r * sin_val
            img = torch.stack([img_r.clamp(0, 1), g, img_b.clamp(0, 1)])

        return img.clamp(0, 1)

# 数据加载

def load_precomputed_data(data_dir: str):
    """从 .pt 文件加载预计算数据到 RAM。
```

```
    Args:
        data_dir: ./data/train 或 ./data/val 目录

    Returns:
        (images: Tensor[N,3,H,W], labels: Tensor[N])
    """
    images_path = os.path.join(data_dir, "images.pt")
    labels_path = os.path.join(data_dir, "labels.pt")

    if not os.path.exists(images_path) or not os.path.exists(labels_path):
        raise FileNotFoundError(
            f"预计算数据不存在: {data_dir}\n"
            f"请先运行: python preprocess.py"
        )

    images = torch.load(images_path, weights_only=True)
    labels = torch.load(labels_path, weights_only=True)
    print(f"[load] {data_dir}: images={images.shape}, labels={labels.shape}")
    return images, labels


def create_dataloader(images, labels, batch_size, sampler=None, color_jitter=None):
    """创建 DataLoader。"""
    dataset = RAMDataset(images, labels, color_jitter=color_jitter)
    shuffle = sampler is None
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        sampler=sampler,
        shuffle=shuffle,
        num_workers=0,  # 0: 主进程加载 (数据已在内存中，无需多进程)
        pin_memory=True,
        drop_last=False,
    )
    return loader


# 训练与验证

def train_one_epoch(model, loader, criterion, optimizer, device):
    """训练一个 epoch，返回平均 loss。"""
    model.train()
```

```
    total_loss = 0.0
    total_samples = 0

    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * images.size(0)
        total_samples += images.size(0)

    return total_loss / total_samples


@torch.no_grad()
def validate(model, loader, criterion, device):
    """验证阶段，返回各项指标。"""
    model.eval()
    total_loss = 0.0
    total_samples = 0
    all_preds = []
    all_labels = []

    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        outputs = model(images)
        loss = criterion(outputs, labels)

        total_loss += loss.item() * images.size(0)
        total_samples += images.size(0)

        preds = outputs.argmax(dim=1)
        all_preds.append(preds.cpu())
        all_labels.append(labels.cpu())

    all_preds = torch.cat(all_preds)
```

```
    all_labels = torch.cat(all_labels)

    # 计算指标
    correct = (all_preds == all_labels).sum().item()
    acc = correct / len(all_labels)

    # TP, TN, FP, FN
    tp = ((all_preds == 1) & (all_labels == 1)).sum().item()
    tn = ((all_preds == 0) & (all_labels == 0)).sum().item()
    fp = ((all_preds == 1) & (all_labels == 0)).sum().item()
    fn = ((all_preds == 0) & (all_labels == 1)).sum().item()

    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0

    metrics = {
        "loss": total_loss / total_samples,
        "accuracy": acc,
        "recall": recall,
        "precision": precision,
        "specificity": specificity,
        "f1": f1,
    }
    return metrics

# Checkpoint 保存

def save_checkpoint(model, optimizer, scheduler, epoch, metrics, path):
    """保存完整 checkpoint。"""
    checkpoint = {
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "scheduler_state_dict": scheduler.state_dict() if scheduler else None,
        "metrics": metrics,
    }
    torch.save(checkpoint, path)


# 主流程
def main():
```

```
    config = load_config("config.yaml")
    set_seed(config["seed"])

    device = torch.device(config["device"] if torch.cuda.is_available() else "cpu")
    print(f"[train] 设备: {device}")

    # 输出目录
    checkpoint_dir = config["checkpoint_dir"]
    model_save_dir = config["model_save_dir"]
    ensure_dir(checkpoint_dir)
    ensure_dir(model_save_dir)

    # 加载预计算数据
    data_dir = config["output_dir"]
    train_images, train_labels = load_precomputed_data(os.path.join(data_dir, "train"))
    val_images, val_labels = load_precomputed_data(os.path.join(data_dir, "val"))

    print(f"[train] 训练集: {train_images.shape[0]} samples")
    print(f"[train] 验证集: {val_images.shape[0]} samples")

    # 采样器 (平衡两类)
    if config["balance_sampler"]:
        class_counts = train_labels.bincount()
        total = len(train_labels)
        # weight = total / (num_classes * count_per_class)
        weights = total / (2.0 * class_counts.float())
        sample_weights = weights[train_labels]
        sampler = WeightedRandomSampler(
            weights=sample_weights.tolist(),
            num_samples=len(train_labels),
            replacement=True,
        )
        print(f"[train] WeightedRandomSampler: 0_weight={weights[0]:.2f},
1_weight={weights[1]:.2f}")
    else:
        sampler = None

    # DataLoader
    cj_gpu = ColorJitterGPU(
        brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05
    )
    train_loader = create_dataloader(
        train_images, train_labels,
```

```
        batch_size=config["batch_size"],
        sampler=sampler,
        color_jitter=cj_gpu,
)
val_loader = create_dataloader(
        val_images, val_labels,
        batch_size=config["batch_size"],
        color_jitter=None,   # 验证集不做 ColorJitter
)

# 模型
model = create_model(
        backbone=config["backbone"],
        num_classes=config["num_classes"],
        pretrained=config["pretrained"],
        dropout=config["dropout"],
)
model = model.to(device)

# 损失函数
alpha = config.get("focal_alpha", [1.0, 2.0])
criterion = FocalLoss(alpha=alpha, gamma=config["focal_gamma"], reduction="mean")

# 优化器: SGD + Momentum
optimizer = optim.SGD(
        model.parameters(),
        lr=config["lr"],
        momentum=config["momentum"],
        weight_decay=config["weight_decay"],
        nesterov=False,
)

# 学习率调度
scheduler = optim.lr_scheduler.StepLR(
        optimizer,
        step_size=config["scheduler_step"],
        gamma=config["scheduler_gamma"],
)

# 训练循环
num_epochs = config["epochs"]
save_every = config["save_every_n_epochs"]
best_val_acc = 0.0
```

```
best_epoch = 0

print(f"\n{'='*60}")
print(f"    训练开始 — {num_epochs} epochs, batch_size={config['batch_size']}")
print(f"    Checkpoint: {checkpoint_dir}")
print(f"    最终模型:     {model_save_dir}")
print(f"{'='*60}\n")

for epoch in range(1, num_epochs + 1):
    epoch_start = time.time()

    # 训练
    train_loss = train_one_epoch(model, train_loader, criterion, optimizer, device)

    # 验证
    val_metrics = validate(model, val_loader, criterion, device)
    val_loss = val_metrics["loss"]
    val_acc = val_metrics["accuracy"]

    # 学习率调度
    scheduler.step()
    current_lr = scheduler.get_last_lr()[0]

    # 日志
    elapsed = time.time() - epoch_start
    print(
        f"Epoch {epoch:3d}/{num_epochs} | "
        f"T_loss={train_loss:.4f} | "
        f"V_loss={val_loss:.4f} | "
        f"V_acc={val_acc:.4f} | "
        f"recall={val_metrics['recall']:.4f} | "
        f"prec={val_metrics['precision']:.4f} | "
        f"spec={val_metrics['specificity']:.4f} | "
        f"lr={current_lr:.6f} | "
        f"{elapsed:.1f}s"
    )

    # 每 save_every_n_epochs 保存 checkpoint
    if epoch % save_every == 0:
        ckpt_path = os.path.join(checkpoint_dir, f"checkpoint_epoch_{epoch:03d}.pth")
        save_checkpoint(model, optimizer, scheduler, epoch, val_metrics, ckpt_path)
        print(f"    -> Checkpoint saved: {ckpt_path}")
```

```
            # 跟踪最佳模型
            if val_acc > best_val_acc:
                best_val_acc = val_acc
                best_epoch = epoch
                best_path = os.path.join(model_save_dir, "best_model.pth")
                save_checkpoint(model, optimizer, scheduler, epoch, val_metrics, best_path)

        # 训练完成
        print(f"\n{'='*60}")
        print(f"  训练完成！")
        print(f"  最佳 Val Acc: {best_val_acc:.4f} @ epoch {best_epoch}")
        print(f"  最佳模型:      {model_save_dir}/best_model.pth")
        print(f"{'='*60}")

        print_metrics(
            {**val_metrics, "best_val_acc": best_val_acc, "best_epoch": float(best_epoch)},
            title="最终验证指标",
        )


if __name__ == "__main__":
    main()
```

inference.py:

```
"""模型推理程序

推理流程:
  1. 加载训练好的模型
  2. 读取输入图片及对应标注文件
  3. bbox 裁剪瓶体 → resize → 边缘增强 → 中心裁剪
  4. 模型推理 → softmax 置信度
  5. 输出预测结果 ("有" / "无") 及两类置信度

用法:
  python inference.py --image 01.jpg --label 01.txt
  python inference.py --help
"""

import os
import sys
import argparse
```

```
import numpy as np
import torch
import torch.nn.functional as F
import torchvision.transforms.functional as TF
from PIL import Image

from model import create_model
from utils import (
    load_config,
    crop_bottle,
    apply_edge_enhance,
    print_inference_result,
)


# 模型加载

def load_model(config: dict, device: torch.device) -> torch.nn.Module:
    model = create_model(
        backbone=config["backbone"],
        num_classes=config["num_classes"],
        pretrained=False,  # 推理时不下载预训练权重，从 checkpoint 加载
        dropout=config["dropout"],
    )

    model_path = config["model_path"]
    if not os.path.exists(model_path):
        raise FileNotFoundError(
            f"模型文件不存在: {model_path}\n"
            f"请先运行: python train.py"
        )

    checkpoint = torch.load(model_path, map_location=device, weights_only=True)
    model.load_state_dict(checkpoint["model_state_dict"])
    model = model.to(device)
    model.eval()

    print(f"[inference] 模型已加载: {model_path}")
    print(f"[inference] 来源 epoch: {checkpoint.get('epoch', 'unknown')}")
    return model
```

```
# 图像预处理

def preprocess_image(
    image_path: str,
    label_path: str,
    config: dict,
) -> torch.Tensor:
    # 完整预处理管道：读取 → 裁剪瓶体 → resize → 边缘增强 → 中心裁剪 → 转 tensor。
    # 读取标注
    with open(label_path, "r") as f:
        line = f.read().strip()
        if not line:
            raise ValueError(f"标注文件为空: {label_path}")
        parts = line.split()
        if len(parts) < 5:
            raise ValueError(f"标注格式无效: {label_path}")
        cls_id = int(parts[0])
        cx, cy, w, h = map(float, parts[1:5])

    # 读取图片
    if not os.path.exists(image_path):
        raise FileNotFoundError(f"图片不存在: {image_path}")
    img = Image.open(image_path).convert("RGB")
    img_np = np.array(img)

    # 裁剪瓶体区域
    bbox = [cls_id, cx, cy, w, h]
    cropped = crop_bottle(img_np, bbox)

    # Resize 到 base_size
    base_size = config["base_size"]
    cropped_pil = Image.fromarray(cropped)
    resized = cropped_pil.resize((base_size, base_size), Image.BICUBIC)
    resized_np = np.array(resized)

    # 边缘增强 (与训练时保持一致)
    edge_method = config.get("edge_enhance", "none")
    edge_kwargs = {
        "clahe_clip_limit": config.get("clahe_clip_limit", 2.0),
        "clahe_grid_size": config.get("clahe_grid_size", 8),
        "unsharp_radius": config.get("unsharp_radius", 1),
        "unsharp_percent": config.get("unsharp_percent", 50),
    }
```

```
    enhanced = apply_edge_enhance(resized_np, method=edge_method, **edge_kwargs)

    # 转为 tensor 并中心裁剪
    img_tensor = TF.to_tensor(enhanced)    # [3, base_size, base_size]

    crop_size = config.get("crop_size", base_size)
    if crop_size < base_size:
        img_tensor = TF.center_crop(img_tensor, crop_size)

    # 添加 batch 维度
    img_tensor = img_tensor.unsqueeze(0)    # [1, 3, crop_size, crop_size]

    return img_tensor

# 推理

@torch.no_grad()
def predict(
    model: torch.nn.Module,
    image_tensor: torch.Tensor,
    device: torch.device,
) -> tuple:
    #执行推理，返回预测类别和置信度。

    image_tensor = image_tensor.to(device)
    logits = model(image_tensor)    # [1, 2]
    probs = F.softmax(logits, dim=1)    # [1, 2]
    conf_empty = probs[0, 0].item()
    conf_has = probs[0, 1].item()
    pred_class = 1 if conf_has > conf_empty else 0

    return pred_class, conf_empty, conf_has


# 主流程

def main():
    parser = argparse.ArgumentParser(
        description="饮料瓶液体有无识别 — 推理程序"
    )
    parser.add_argument(
        "--image", type=str, required=True,
        help="输入图片路径 (e.g. 01.jpg)"
```

```
    )
    parser.add_argument(
        "--label", type=str, required=True,
        help="YOLO 标注文件路径 (e.g. 01.txt)"
    )
    parser.add_argument(
        "--config", type=str, default="config.yaml",
        help="配置文件路径 (default: config.yaml)"
    )
    parser.add_argument(
        "--model", type=str, default=None,
        help="模型文件路径 (覆盖 config 中的 model_path)"
    )
    args = parser.parse_args()

    # 加载配置
    config = load_config(args.config)
    device = torch.device(config["device"] if torch.cuda.is_available() else "cpu")

    # 允许命令行覆盖模型路径
    if args.model:
        config["model_path"] = args.model

    # 加载模型
    model = load_model(config, device)

    # 预处理
    image_tensor = preprocess_image(args.image, args.label, config)

    # 推理
    pred_class, conf_empty, conf_has = predict(model, image_tensor, device)

    # 输出结果
    print_inference_result(pred_class, conf_empty, conf_has)


if __name__ == "__main__":
    main()
```

train_yolo_4class.py:

```
"""YOLO11n 4 类检测训练 — empty/little/middle/much (低内存优化版)
```

直接用 YOLO 检测模型在原始 4 类标注上训练，同时完成瓶体检测和液量分类。

用法:
  python train_yolo_4class.py --dir ./test --label_dir ./test
"""

```
import os
import glob
import yaml
from datetime import datetime, timezone, timedelta
from collections import Counter

from ultralytics import YOLO
from sklearn.model_selection import train_test_split

from utils import load_config, set_seed, ensure_dir


def get_timestamp():
    """生成北京时间戳字符串。"""
    tz = timezone(timedelta(hours=8))
    return datetime.now(tz).strftime("%Y%m%d_%H%M%S")


def _link_or_copy(src, dst):
    """尝试硬链接，失败则回退到复制（Windows 兼容）。"""
    try:
        os.link(src, dst)
    except (OSError, PermissionError, NotImplementedError):
        import shutil
        shutil.copy2(src, dst)


def prepare_yolo_dataset(source_dir, output_dir, train_ratio=0.8, seed=42):
    """将 4 类 YOLO 标注数据构建为训练目录结构（优化版）。

    使用硬链接代替文件复制，减少磁盘占用和 IO 时间。
    """
    src_images = os.path.join(source_dir, "images")
    src_labels = os.path.join(source_dir, "labels")

    if not os.path.exists(src_images):
```

```
        raise FileNotFoundError(f"图片目录不存在: {src_images}")
if not os.path.exists(src_labels):
        raise FileNotFoundError(f"标注目录不存在: {src_labels}")

# 收集有效图片-标注对（一次遍历，不存额外中间结构）
valid_images = []      # [(stem, img_path)]
class_labels = []      # [cls_id]

for ext in ["*.jpg", "*.png", "*.jpeg", "*.JPG", "*.JPEG", "*.PNG"]:
        for f in glob.glob(os.path.join(src_images, ext)):
                stem = os.path.splitext(os.path.basename(f))[0]
                label_path = os.path.join(src_labels, f"{stem}.txt")
                if not os.path.exists(label_path):
                        continue
                with open(label_path, "r") as lf:
                        content = lf.read().strip()
                        if content:
                                cls_id = int(content.split()[0])
                                if 0 <= cls_id <= 3:
                                        valid_images.append((stem, f))
                                        class_labels.append(cls_id)

print(f"找到 {len(valid_images)} 对有效图片-标注")

if len(valid_images) == 0:
        raise RuntimeError("未找到有效的图片-标注对")

# 分层划分
train_indices, val_indices = train_test_split(
        range(len(valid_images)),
        test_size=1.0 - train_ratio,
        stratify=class_labels,
        random_state=seed,
        shuffle=True,
)

# 创建目录结构并链接文件
for sub in ["images/train", "images/val", "labels/train", "labels/val"]:
        ensure_dir(os.path.join(output_dir, sub))

for idx_list, split_name in [(train_indices, "train"), (val_indices, "val")]:
        for idx in idx_list:
                stem, img_path = valid_images[idx]
```

```
            img_ext = os.path.splitext(img_path)[1]

            dst_img = os.path.join(output_dir, "images", split_name, f"{stem}{img_ext}")
            _link_or_copy(img_path, dst_img)

            src_lbl = os.path.join(src_labels, f"{stem}.txt")
            dst_lbl = os.path.join(output_dir, "labels", split_name, f"{stem}.txt")
            _link_or_copy(src_lbl, dst_lbl)

    # 打印分布
    for split_name, idx_list in [("train", train_indices), ("val", val_indices)]:
        counts = Counter([class_labels[i] for i in idx_list])
        print(f"\n{split_name} 集分布 ({len(idx_list)} 张):")
        for c, name in enumerate(["empty", "little", "middle", "much"]):
            print(f"  {c} ({name}): {counts.get(c, 0)}")

    # 生成 dataset.yaml
    dataset_yaml = {
        "path": os.path.abspath(output_dir),
        "train": "images/train",
        "val": "images/val",
        "names": {0: "empty", 1: "little", 2: "middle", 3: "much"},
        "nc": 4,
    }

    yaml_path = os.path.join(output_dir, "dataset.yaml")
    with open(yaml_path, "w", encoding="utf-8") as f:
        yaml.dump(dataset_yaml, f, default_flow_style=False, allow_unicode=True)

    print(f"\ndataset.yaml 已保存到: {yaml_path}")
    return yaml_path


def main():
    config = load_config("config.yaml")
    set_seed(config.get("seed", 42))

    model_save_dir = config.get("model_save_dir", "./output/models")
    ensure_dir(model_save_dir)

    timestamp = get_timestamp()
    source_dir = "datast_latest"
    dataset_dir = "data_yolo_4class"
```

```
# 1. 准备 YOLO 数据集
print("=" * 70)
print("    Step 1: 准备 YOLO 4 类检测数据集")
print("=" * 70)
dataset_yaml = prepare_yolo_dataset(source_dir, dataset_dir)

# 2. 加载预训练模型
print(f"\n{'='*70}")
print(f"    Step 2: 训练 YOLO11n 4 类检测模型 (RAM 优化模式)")
print(f"{'='*70}")

local_pt = "yolo11n.pt"
if os.path.exists(local_pt):
    model = YOLO(local_pt)
    print(f"使用本地模型: {local_pt}")
else:
    model = YOLO("yolo11n.pt")
    print("使用在线预训练模型: yolo11n.pt")

# 3. 训练（RAM 优化参数）
results = model.train(
    data=dataset_yaml,
    epochs=80,
    imgsz=640,
    batch=24,                    # 32→16，减少批次内存
    lr0=0.01,
    lrf=0.01,
    momentum=0.937,
    weight_decay=0.0005,
    warmup_epochs=3,
    cos_lr=True,
    close_mosaic=15,
    workers=2,                   # 4→1，单进程加载，大幅降低 RAM
    amp=True,                    # 混合精度，减少显存占用
    cache="disk",                # 预处理缓存至磁盘，避免重复 JPEG 解码
    rect=True,                   # 矩形训练，减少 padding 内存/计算
    device=0,
    project="runs",
    name=f"yolo_4class_{timestamp}",
    exist_ok=True,
    plots=False,                 # 禁用绘图，省 matplotlib RAM
    save=True,
```

```
        val=True,
    )

    # 4. 复制最佳模型到 output/models/
    import shutil
    best_pt = os.path.join("runs","detect","runs",f"yolo_4class_{timestamp}", "weights", "best.pt")
    if os.path.exists(best_pt):
        dst = os.path.join(model_save_dir, f"yolo_4class_{timestamp}.pt")
        shutil.copy2(best_pt, dst)
        print(f"\n 最佳模型已保存: {dst}")
    else:
        print(f"\n[WARNING] 最佳模型文件未找到: {best_pt}")

    print(f"\n 训练完成!")


if __name__ == "__main__":
    main()
```

Inference_yolo_4class.py:

```
"""YOLO11n 4 类检测推理 + 可视化 — empty/little/middle/much

直接用 YOLO 模型同时完成瓶体检测和液量分类。
相比两阶段管道:
  - 无需 Stage 1 检测 + Stage 2 分类的两步流程
  - 单一模型文件, 推理更快
  - YOLO 输出直接包含 bbox + cls + confidence

可视化:
  --visualize: 在原图上绘制检测框 (带类别标签和置信度)
  --vis_dir:   指定可视化输出目录 (默认 ./output/vis_yolo/)

用法:
  # 单张推理 + 可视化
  python inference_yolo_4class.py --image ./test/01.jpg --visualize
  # 批量评估 + 可视化
  python inference_yolo_4class.py --dir ./test/ --visualize
"""

import os
import glob
```

```
import sys
import argparse
from collections import Counter

import numpy as np
import cv2
from ultralytics import YOLO
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
)

from utils import load_config, set_seed, ensure_dir


# 可视化常量

CLASS_NAMES_EN = ["empty", "little", "middle", "much"]
CLASS_NAMES_CN = ["空", "少", "中", "多"]

# 4 类标签对应的颜色 (BGR)
CLASS_COLORS = {
    0: (0, 0, 255),       # empty -> 红色
    1: (0, 165, 255),     # little -> 橙色
    2: (0, 255, 255),     # middle -> 黄色
    3: (0, 255, 0),       # much -> 绿色
}


def draw_detections(
    image_bgr: np.ndarray,
    boxes: np.ndarray,
    class_ids: np.ndarray,
    confidences: np.ndarray,
    gt_box=None,
    gt_label=None,
    output_path: str = None,
    show: bool = False,
    show_delay: int = 0,
):
    # 在原图上绘制 YOLO 检测结果 (带信息面板).
    h, w = image_bgr.shape[:2]
```

```
# 绘制所有检测框
for i in range(len(boxes)):
    x1, y1, x2, y2 = map(int, boxes[i])
    cls_id = int(class_ids[i])
    conf = float(confidences[i])

    color = CLASS_COLORS.get(cls_id, (255, 255, 255))
    thickness = 3 if i == 0 else 1    # 最高置信度的用粗框

    cv2.rectangle(image_bgr, (x1, y1), (x2, y2), color, thickness)

    label = f"{CLASS_NAMES_EN[cls_id]}({CLASS_NAMES_CN[cls_id]}) {conf:.2f}"
    # 背景文字
    (tw, th), baseline = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
    cv2.rectangle(image_bgr,
                  (x1, y1 - th - baseline - 8),
                  (x1 + tw + 4, y1),
                  color, -1)
    cv2.putText(image_bgr, label,
                (x1 + 2, y1 - baseline - 4),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)

#左侧信息面板
y_offset = 30
line_height = 28

# 背景半透明遮罩
overlay = image_bgr.copy()
panel_w = 360
num_lines = 3 + min(len(boxes), 5) * 1
if gt_label is not None:
    num_lines += 3
panel_h = line_height * num_lines + 20
cv2.rectangle(overlay, (5, 5), (5 + panel_w, 5 + panel_h), (40, 40, 40), -1)
cv2.addWeighted(overlay, 0.6, image_bgr, 0.4, 0, image_bgr)

# 检测数量
cv2.putText(image_bgr, f"Detections: {len(boxes)}",
            (15, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
            (255, 255, 255), 2)
y_offset += line_height
```

```
    # 显示前 5 个检测
    top_n = min(len(boxes), 5)
    for i in range(top_n):
        cls_id = int(class_ids[i])
        conf = float(confidences[i])
        color = CLASS_COLORS.get(cls_id, (255, 255, 255))
        cv2.putText(image_bgr,
                    f"  #{i+1}:
{CLASS_NAMES_EN[cls_id]:>6s}({CLASS_NAMES_CN[cls_id]}) conf={conf:.3f}",
                    (15, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
        y_offset += line_height

    # 分隔线
    y_offset += 4
    cv2.line(image_bgr, (15, y_offset), (15 + panel_w - 30, y_offset),
             (150, 150, 150), 1)
    y_offset += line_height


    if gt_label is not None:
        gt_name = CLASS_NAMES_EN[gt_label]
        gt_color = CLASS_COLORS.get(gt_label, (255, 255, 255))

        cv2.putText(image_bgr, f"GT: {gt_name} ({CLASS_NAMES_CN[gt_label]})",
                    (15, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.6, gt_color, 2)
        y_offset += line_height

        # 判断匹配
        match_str = ""
        match_color = (200, 200, 200)
        if len(class_ids) > 0:
            pred_cls = int(class_ids[0])
            if pred_cls == gt_label:
                match_str = "MATCH ✓"
                match_color = (0, 255, 0)
            else:
                match_str = f"MISMATCH ✗ (pred={CLASS_NAMES_CN[pred_cls]})"
                match_color = (0, 0, 255)
        else:
            match_str = "MISS (未检出)"
            match_color = (0, 0, 255)

        cv2.putText(image_bgr, f"  -> {match_str}",
```

```
                    (15, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.55, match_color, 2)
    else:
        cv2.putText(image_bgr, "GT: (无标注)",
                    (15, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (180, 180, 180), 1)

    if output_path:
        ensure_dir(os.path.dirname(output_path))
        cv2.imwrite(output_path, image_bgr)
        print(f"   [vis] 保存: {output_path}")

    if show:
        win_name = os.path.basename(output_path) if output_path else "detection"
        cv2.namedWindow(win_name, cv2.WINDOW_NORMAL)
        cv2.resizeWindow(win_name, min(w, 1280), min(h, 900))
        cv2.imshow(win_name, image_bgr)
        print(f"   [vis] 弹窗显示中...")
        key = cv2.waitKey(show_delay)
        cv2.destroyWindow(win_name)


def predict_single(model, image_path: str, conf_threshold: float = 0.25, verbose: bool = True):
    """对单张图片进行 YOLO 4 类推理.

    Returns:
        dict: {
            "image": str,
            "detections": [(cls_id, conf, x1, y1, x2, y2), ...],
            "top_pred": cls_id or -1,
            "top_conf": float,
        }
    """
    results = model(image_path, conf=conf_threshold, verbose=False)

    if len(results) == 0 or results[0].boxes is None or len(results[0].boxes) == 0:
        if verbose:
            print(f"\n{'='*60}")
            print(f"   图片: {os.path.basename(image_path)}")
            print(f"   未检测到任何瓶体!")
            print(f"{'='*60}")
        return {
            "image": image_path,
            "detections": [],
            "top_pred": -1,
```

```
                "top_conf": 0.0,
                "boxes": np.array([]),
                "class_ids": np.array([]),
                "confidences": np.array([]),
            }

        boxes = results[0].boxes
        xyxy = boxes.xyxy.cpu().numpy()
        class_ids = boxes.cls.cpu().numpy().astype(int)
        confidences = boxes.conf.cpu().numpy()

        # 按置信度排序 (YOLO 默认已排序, 保险再做一次)
        order = np.argsort(confidences)[::-1]
        xyxy = xyxy[order]
        class_ids = class_ids[order]
        confidences = confidences[order]

        detections = []
        for i in range(len(xyxy)):
            detections.append((
                int(class_ids[i]),
                float(confidences[i]),
                float(xyxy[i][0]),
                float(xyxy[i][1]),
                float(xyxy[i][2]),
                float(xyxy[i][3]),
            ))

        top_pred = int(class_ids[0]) if len(class_ids) > 0 else -1
        top_conf = float(confidences[0]) if len(confidences) > 0 else 0.0

        if verbose:
            print(f"\n{'='*60}")
            print(f"  图片: {os.path.basename(image_path)}")
            print(f"  检测到 {len(detections)} 个瓶体")
            for i, (cls_id, conf, x1, y1, x2, y2) in enumerate(detections[:5]):
                name = CLASS_NAMES_EN[cls_id]
                name_cn = CLASS_NAMES_CN[cls_id]
                print(f"    [{i+1}] {name}({name_cn})  conf={conf:.4f}
box=({x1:.0f},{y1:.0f},{x2:.0f},{y2:.0f})")
            print(f"  最终预测: {CLASS_NAMES_EN[top_pred] if top_pred >= 0 else 'none'}")
            print(f"{'='*60}")
```

```
    return {
        "image": image_path,
        "detections": detections,
        "top_pred": top_pred,
        "top_conf": top_conf,
        "boxes": xyxy,
        "class_ids": class_ids,
        "confidences": confidences,
    }


def evaluate_dataset(
    model,
    img_dir: str,
    label_dir: str,
    conf_threshold: float = 0.25,
    vis_dir: str = None,
):
    """在带标注的数据集上评估 YOLO 4 类模型.

    Args:
        img_dir: 图片目录
        label_dir: YOLO 标注目录
        conf_threshold: 检测置信度阈值
        vis_dir: 可视化输出目录 (None = 不可视化)
    """
    image_files = []
    for ext in ["*.jpg", "*.png", "*.jpeg", "*.JPG", "*.JPEG", "*.PNG"]:
        image_files.extend(glob.glob(os.path.join(img_dir, ext)))
    image_files.sort()

    if not image_files:
        print(f"[ERROR] 未找到图片: {img_dir}")
        return

    print(f"\n{'='*70}")
    print(f"  YOLO 4 类评估")
    print(f"  图片目录: {img_dir}")
    print(f"  标注目录: {label_dir}")
    print(f"  图片数量: {len(image_files)}")
    if vis_dir:
        print(f"  可视化:   {vis_dir}/")
    print(f"{'='*70}")
```

```
y_true = []
y_pred = []
misses = 0

for img_path in image_files:
    stem = os.path.splitext(os.path.basename(img_path))[0]

    # 读取 GT 标注
    label_path = os.path.join(label_dir, f"{stem}.txt")
    gt_label = None
    gt_box = None
    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            lines = f.read().strip().splitlines()
            if lines and lines[0].strip():
                parts = lines[0].strip().split()
                if len(parts) >= 5:
                    gt_label = int(parts[0])
                    gt_box = tuple(map(float, parts[1:5]))

    # 推理
    result = predict_single(model, img_path, conf_threshold=conf_threshold, verbose=False)

    if result["top_pred"] < 0:
        misses += 1
        y_pred.append(-1)
        y_true.append(gt_label if gt_label is not None else -1)
    else:
        y_pred.append(result["top_pred"])
        y_true.append(gt_label if gt_label is not None else -1)

    # 可视化
    if vis_dir:
        img_bgr = cv2.imread(img_path)
        if img_bgr is not None:
            vis_path = os.path.join(vis_dir, f"{stem}.jpg")
            draw_detections(
                img_bgr,
                result["boxes"],
                result["class_ids"],
                result["confidences"],
                gt_box=gt_box,
```

```
                gt_label=gt_label,
                output_path=vis_path,
                show=True,
                show_delay=500,
            )

# 过滤有效配对
valid_mask = [(yt >= 0 and yp >= 0) for yt, yp in zip(y_true, y_pred)]
filtered_yt = [yt for yt, m in zip(y_true, valid_mask) if m]
filtered_yp = [yp for yp, m in zip(y_pred, valid_mask) if m]

print(f"\n   未检出: {misses}")
print(f"   有效配对: {len(filtered_yt)}")

if len(filtered_yt) == 0:
    print("   [WARNING] 无可评估样本")
    return

acc = accuracy_score(filtered_yt, filtered_yp)
precision_macro, recall_macro, f1_macro, _ = precision_recall_fscore_support(
    filtered_yt, filtered_yp, average="macro", zero_division=0
)
precision_w, recall_w, f1_w, _ = precision_recall_fscore_support(
    filtered_yt, filtered_yp, average="weighted", zero_division=0
)
precision_per, recall_per, f1_per, support_per = precision_recall_fscore_support(
    filtered_yt, filtered_yp, average=None, zero_division=0, labels=[0, 1, 2, 3]
)
cm = confusion_matrix(filtered_yt, filtered_yp, labels=[0, 1, 2, 3])

print(f"\n{'='*70}")
print(f"   评估结果")
print(f"{'='*70}")
print(f"   样本数: {len(filtered_yt)} (+{misses} 未检出)")
print(f"   准确率: {acc:.4f} ({acc*100:.1f}%)")
print(f"   宏平均    P={precision_macro:.4f}   R={recall_macro:.4f}   F1={f1_macro:.4f}")
print(f"   加权平均  P={precision_w:.4f}   R={recall_w:.4f}   F1={f1_w:.4f}")
print()
print(f"   每类指标:")
for c in range(4):
    prec = precision_per[c] if c < len(precision_per) else 0
    rec = recall_per[c] if c < len(recall_per) else 0
    f1 = f1_per[c] if c < len(f1_per) else 0
```

```
            supp = support_per[c] if c < len(support_per) else 0
            print(f"       {CLASS_NAMES_EN[c]:>8s}: prec={prec:.4f}, rec={rec:.4f}, f1={f1:.4f},
supp={supp}")

    print(f"\n   混淆矩阵:")
    print(f"                  {' '.join(f'{n:>6s}' for n in CLASS_NAMES_EN)}")
    for i in range(4):
        row = cm[i] if i < len(cm) else [0, 0, 0, 0]
        print(f"       {CLASS_NAMES_EN[i]:>6s}: {' '.join(f'{v:6d}' for v in row)}")


def main():
    parser = argparse.ArgumentParser(
        description="YOLO11n 4 类检测推理 + 可视化 (empty/little/middle/much)"
    )
    parser.add_argument("--image", type=str, help="单张图片路径")
    parser.add_argument("--dir", type=str, help="图片目录路径")
    parser.add_argument("--label_dir", type=str, help="YOLO 标注目录 (用于评估)")
    parser.add_argument("--model", type=str, default="./output/models/best.pt", help="YOLO 模型路径
(.pt)")
    parser.add_argument("--config", type=str, default="config.yaml",
                        help="配置文件路径")
    parser.add_argument("--conf", type=float, default=0.25,
                        help="检测置信度阈值")
    parser.add_argument("--visualize", action="store_true",
                        help="生成可视化图片")
    parser.add_argument("--vis_dir", type=str, default=None,
                        help="可视化输出目录 (默认 ./output/vis_yolo/)")

    args = parser.parse_args()

    if not args.image and not args.dir:
        parser.print_help()
        print("\n[ERROR] 请指定 --image 或 --dir")
        sys.exit(1)

    config = load_config(args.config)
    set_seed(config.get("seed", 42))
    model_save_dir = config.get("model_save_dir", "./output/models")

    # 确定模型路径
    if args.model:
        model_path = args.model
```

```
    else:
        candidates = sorted(glob.glob(os.path.join(model_save_dir, "best.pt")))
        if not candidates:
            print(f"[ERROR] 未找到 YOLO 4 类模型. 请指定 --model 或先运行
train_yolo_4class.py")
            sys.exit(1)
        model_path = candidates[-1]

    # 加载模型
    print(f"加载模型: {model_path}")
    model = YOLO(model_path)

    # 可视化目录
    vis_dir = None
    if args.visualize:
        vis_dir = args.vis_dir or "./output/vis_yolo"
        ensure_dir(vis_dir)

    # 推理
    if args.image:
        result = predict_single(model, args.image, args.conf, verbose=True)

        if args.visualize and result["boxes"].size > 0:
            stem = os.path.splitext(os.path.basename(args.image))[0]
            img_bgr = cv2.imread(args.image)
            if img_bgr is not None:
                vis_path = os.path.join(vis_dir, f"{stem}.jpg")
                draw_detections(
                    img_bgr,
                    result["boxes"],
                    result["class_ids"],
                    result["confidences"],
                    output_path=vis_path,
                    show=True,
                    show_delay=0,
                )

    elif args.dir:
        label_dir = args.label_dir if args.label_dir else args.dir
        evaluate_dataset(model, args.dir, label_dir, args.conf, vis_dir)


if __name__ == "__main__":
```

```
    main()
```

evaluate_task2.py:

```
"""评估脚本 — 二分类（有/无液体）模型批量评估


评估流程:
  1. 加载训练好的二分类模型（ResNet backbone）
  2. 遍历图片目录，读取对应 YOLO 标注文件获取 bbox 和 GT 类别
  3. GT 映射: class 0 (empty) → 0 (无), class 1/2/3 (little/middle/much) → 1 (有)
  4. 使用与 inference.py 相同的预处理管道 → 模型推理 → softmax
  5. 计算二分类指标: accuracy, precision, recall, f1, confusion matrix

用法:
  python evaluate_task2.py --dir ./test --label_dir ./test --model ./output/models/best.pth
  python evaluate_task2.py --dir ./datast_latest/images --label_dir ./datast_latest/labels --
model ./output/models/best_model.pth --config config.yaml
"""

import os
import glob
import argparse
import numpy as np
import torch
import torch.nn.functional as F
import torchvision.transforms.functional as TF
from PIL import Image
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    classification_report,
)

from model import create_model
from utils import (
    load_config,
    crop_bottle,
    apply_edge_enhance,
)
```

```
CLASS_NAMES = ["empty", "has"]
CLASS_NAMES_CN = ["无", "有"]


def load_model_from_config(config: dict, device: torch.device) -> torch.nn.Module:
    """加载训练好的二分类模型。"""
    model = create_model(
        backbone=config["backbone"],
        num_classes=config["num_classes"],
        pretrained=False,
        dropout=config["dropout"],
    )

    model_path = config["model_path"]
    if not os.path.exists(model_path):
        raise FileNotFoundError(
            f"模型文件不存在: {model_path}\n 请先运行: python train.py"
        )

    checkpoint = torch.load(model_path, map_location=device, weights_only=True)
    model.load_state_dict(checkpoint["model_state_dict"])
    model = model.to(device)
    model.eval()

    print(f"[evaluate] 模型已加载: {model_path}")
    print(f"[evaluate] 来源 epoch: {checkpoint.get('epoch', 'unknown')}")
    return model


def map_class_to_binary(cls_id: int) -> int:
    """将原始 4 类标签映射为二元：
    0 (empty) → 0 (无液体)
    1 (little) / 2 (middle) / 3 (much) → 1 (有液体)
    """
    return 0 if cls_id == 0 else 1


def preprocess_single(image_path: str, label_path: str, config: dict) -> torch.Tensor:
    """与 inference.py 相同的预处理管道。

    流程: 读取图片 → 读取 YOLO label (获取 bbox) → 裁剪瓶体 → resize
          → 边缘增强 → 中心裁剪 → 转 tensor → 添加 batch 维度
```

```
Returns:
    [1, 3, crop_size, crop_size]  图像张量
"""
# 读取标注获取 bbox
with open(label_path, "r") as f:
    line = f.read().strip()
    if not line:
        raise ValueError(f"标注文件为空: {label_path}")
    parts = line.split()
    if len(parts) < 5:
        raise ValueError(f"标注格式无效: {label_path}")
    cls_id = int(parts[0])
    cx, cy, w, h = map(float, parts[1:5])

# 读取图片
if not os.path.exists(image_path):
    raise FileNotFoundError(f"图片不存在: {image_path}")
img = Image.open(image_path).convert("RGB")
img_np = np.array(img)

# 裁剪瓶体区域
bbox = [cls_id, cx, cy, w, h]
cropped = crop_bottle(img_np, bbox)

# Resize 到 base_size
base_size = config["base_size"]
cropped_pil = Image.fromarray(cropped)
resized = cropped_pil.resize((base_size, base_size), Image.BICUBIC)
resized_np = np.array(resized)

# 边缘增强
edge_method = config.get("edge_enhance", "none")
edge_kwargs = {
    "clahe_clip_limit": config.get("clahe_clip_limit", 2.0),
    "clahe_grid_size": config.get("clahe_grid_size", 8),
    "unsharp_radius": config.get("unsharp_radius", 1),
    "unsharp_percent": config.get("unsharp_percent", 50),
}
enhanced = apply_edge_enhance(resized_np, method=edge_method, **edge_kwargs)

# 转 tensor 并中心裁剪
img_tensor = TF.to_tensor(enhanced)    # [3, base_size, base_size]
```

```
    crop_size = config.get("crop_size", base_size)
    if crop_size < base_size:
        img_tensor = TF.center_crop(img_tensor, crop_size)

    # 添加 batch 维度
    img_tensor = img_tensor.unsqueeze(0)    # [1, 3, crop_size, crop_size]

    return img_tensor


@torch.no_grad()
def predict_single(model: torch.nn.Module, image_tensor: torch.Tensor, device: torch.device):
    """执行推理，返回预测类别和置信度。

    Returns:
        (pred_class, conf_empty, conf_has)
    """
    image_tensor = image_tensor.to(device)
    logits = model(image_tensor)    # [1, 2]
    probs = F.softmax(logits, dim=1)    # [1, 2]
    conf_empty = probs[0, 0].item()
    conf_has = probs[0, 1].item()
    pred_class = 1 if conf_has > conf_empty else 0

    return pred_class, conf_empty, conf_has


def evaluate_binary(model, img_dir, label_dir, config, device):
    """评估二分类（有/无液体）性能。

    遍历所有图片，使用 YOLO 标注获取 bbox 和 GT，
    预处理后推理得到二分类预测，计算二分类指标。
    """
    # 收集所有图片
    image_files = []
    for ext in ["*.jpg", "*.png", "*.jpeg", "*.JPG", "*.JPEG", "*.PNG"]:
        image_files.extend(glob.glob(os.path.join(img_dir, ext)))
    image_files.sort()
    if not image_files:
        print(f"[ERROR] 未找到图片: {img_dir}")
        return

    print(f"\n{'='*70}")
```

```
print(f"   二分类评估：液体有无识别 (2 类)")
print(f"   图片目录: {img_dir}")
print(f"   标注目录: {label_dir}")
print(f"   图片数量: {len(image_files)}")
print(f"{'='*70}")

y_true = []
y_pred = []
confidences = []   # 存储预测为正类的置信度 (conf_has)
error_count = 0   # 预处理失败的样本数

for img_path in image_files:
    stem = os.path.splitext(os.path.basename(img_path))[0]
    label_path = os.path.join(label_dir, f"{stem}.txt")

    # 读取 GT
    gt_class = None
    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            line = f.readline().strip()
            if line:
                parts = line.split()
                if len(parts) >= 5:
                    gt_class_raw = int(parts[0])
                    gt_class = map_class_to_binary(gt_class_raw)

    if gt_class is None:
        # 无有效 GT，跳过
        error_count += 1
        continue

    # 预处理 + 推理
    try:
        image_tensor = preprocess_single(img_path, label_path, config)
        pred_class, conf_empty, conf_has = predict_single(model, image_tensor, device)
        y_true.append(gt_class)
        y_pred.append(pred_class)
        confidences.append(conf_has)
    except Exception as e:
        print(f"[WARN] 处理 {stem} 时出错: {e}")
        error_count += 1
        continue
```

```
    # 统计
    total_valids = len(y_true)
    print(f"\n   预处理失败 / 无标注图片数: {error_count}")
    print(f"   有效评估样本数: {total_valids}")

    if total_valids == 0:
        print("   无有效样本，退出")
        return

    # 计算指标
    acc = accuracy_score(y_true, y_pred)
    precision, recall, f1, support = precision_recall_fscore_support(
        y_true, y_pred, labels=[0, 1], zero_division=0
    )
    # 宏平均
    precision_macro, recall_macro, f1_macro, _ = precision_recall_fscore_support(
        y_true, y_pred, average="macro", zero_division=0
    )
    # 加权平均
    precision_weighted, recall_weighted, f1_weighted, _ = precision_recall_fscore_support(
        y_true, y_pred, average="weighted", zero_division=0
    )
    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])

    # 输出结果
    print(f"\n{'='*70}")
    print(f"   二分类评估结果（液体有无识别）")
    print(f"{'='*70}")
    print(f"总体准确率 (Accuracy): {acc:.4f} ({acc*100:.1f}%)")
    print(f"\n 宏平均   P={precision_macro:.4f}   R={recall_macro:.4f}   F1={f1_macro:.4f}")
    print(f"加权平均  P={precision_weighted:.4f}   R={recall_weighted:.4f}   F1={f1_weighted:.4f}")

    print(f"\n 每类指标:")
    for i, name in enumerate(CLASS_NAMES):
        print(f"   {name:>8s}: prec={precision[i]:.4f}, rec={recall[i]:.4f}, f1={f1[i]:.4f},
supp={support[i]}")

    print(f"\n 混淆矩阵 (行:真实, 列:预测):")
    print("              " + " ".join(f"{name:>6s}" for name in CLASS_NAMES))
    for i, name in enumerate(CLASS_NAMES):
        row_str = " ".join(f"{cm[i, j]:6d}" for j in range(2))
        print(f"   {name:>6s}: {row_str}")
```

```
    # 详细分类报告
    print("\n 详细分类报告 (sklearn):")
    print(classification_report(y_true, y_pred, target_names=CLASS_NAMES, zero_division=0))


def main():
    parser = argparse.ArgumentParser(description="评估二分类模型：液体有无识别")
    parser.add_argument("--dir", required=True, help="测试图片目录")
    parser.add_argument("--label_dir", help="YOLO 标注目录（默认同--dir）")
    parser.add_argument("--model", required=True, help="二分类模型路径 (.pth)")
    parser.add_argument("--config", type=str, default="config.yaml", help="配置文件路径 (default:
config.yaml)")
    args = parser.parse_args()

    label_dir = args.label_dir if args.label_dir else args.dir

    # 加载配置
    config = load_config(args.config)
    device = torch.device(config.get("device", "cuda") if torch.cuda.is_available() else "cpu")
    print(f"[evaluate] 使用设备: {device}")

    # 允许命令行覆盖模型路径
    config["model_path"] = args.model

    # 加载模型
    model = load_model_from_config(config, device)

    # 执行批量评估
    evaluate_binary(model, args.dir, label_dir, config, device)


if __name__ == "__main__":
    main()
```

evaluate_task3.py:

```
import os
import glob
import argparse
import numpy as np
from ultralytics import YOLO
from sklearn.metrics import (
```

```
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    classification_report
)

CLASS_NAMES = ["empty", "little", "middle", "much"]
CLASS_NAMES_CN = ["空", "少量", "中等", "较多"]

def evaluate_4class(model, img_dir, label_dir, conf_threshold=0.25):
    """评估四分类（含量分级）性能"""
    # 收集所有图片
    image_files = []
    for ext in ["*.jpg", "*.png", "*.jpeg", "*.JPG", "*.JPEG", "*.PNG"]:
        image_files.extend(glob.glob(os.path.join(img_dir, ext)))
    image_files.sort()
    if not image_files:
        print(f"[ERROR] 未找到图片: {img_dir}")
        return

    print(f"\n{'='*70}")
    print(f"  任务三评估：自动定位 + 液体含量分级 (4 类)")
    print(f"  图片目录: {img_dir}")
    print(f"  标注目录: {label_dir}")
    print(f"  图片数量: {len(image_files)}")
    print(f"{'='*70}")

    y_true = []
    y_pred = []
    miss_count = 0    # 模型未检测到任何瓶子的图片数

    for img_path in image_files:
        stem = os.path.splitext(os.path.basename(img_path))[0]
        # 读取 GT (YOLO 格式)
        label_path = os.path.join(label_dir, f"{stem}.txt")
        gt_class = None
        if os.path.exists(label_path):
            with open(label_path, 'r') as f:
                line = f.readline().strip()
                if line:
                    gt_class = int(line.split()[0])    # 0,1,2,3
        # 模型推理
        results = model(img_path, conf=conf_threshold, verbose=False)
```

```
        if len(results)==0 or results[0].boxes is None or len(results[0].boxes)==0:
            miss_count += 1
            y_pred.append(-1)
            y_true.append(gt_class if gt_class is not None else -1)
        else:
            boxes = results[0].boxes
            # 取置信度最高的预测框作为整图的含量预测
            top_idx = np.argmax(boxes.conf.cpu().numpy())
            pred_class = int(boxes.cls.cpu().numpy()[top_idx])
            y_pred.append(pred_class)
            y_true.append(gt_class if gt_class is not None else -1)

# 过滤有效样本（既有 GT 又有预测）
valid = [(yt>=0 and yp>=0) for yt,yp in zip(y_true, y_pred)]
y_true_valid = [yt for yt,m in zip(y_true, valid) if m]
y_pred_valid = [yp for yp,m in zip(y_pred, valid) if m]

print(f"\n  未检测到瓶体的图片数: {miss_count}")
print(f"  有效评估样本数: {len(y_true_valid)}")

if len(y_true_valid) == 0:
    print("  无有效样本，退出")
    return

# 计算指标
acc = accuracy_score(y_true_valid, y_pred_valid)
precision, recall, f1, support = precision_recall_fscore_support(
    y_true_valid, y_pred_valid, labels=[0,1,2,3], zero_division=0
)
# 宏平均
precision_macro, recall_macro, f1_macro, _ = precision_recall_fscore_support(
    y_true_valid, y_pred_valid, average='macro', zero_division=0
)
# 加权平均
precision_weighted, recall_weighted, f1_weighted, _ = precision_recall_fscore_support(
    y_true_valid, y_pred_valid, average='weighted', zero_division=0
)
cm = confusion_matrix(y_true_valid, y_pred_valid, labels=[0,1,2,3])

# 输出结果
print(f"\n{'='*70}")
print(f"  四分类评估结果（含量分级）")
print(f"{'='*70}")
```

```
    print(f"总体准确率 (Accuracy): {acc:.4f} ({acc*100:.1f}%)")
    print(f"\n 宏平均   P={precision_macro:.4f}   R={recall_macro:.4f}   F1={f1_macro:.4f}")
    print(f"加权平均 P={precision_weighted:.4f}   R={recall_weighted:.4f}   F1={f1_weighted:.4f}")

    print(f"\n 每类指标:")
    for i, name in enumerate(CLASS_NAMES):
        print(f"   {name:>8s}: prec={precision[i]:.4f}, rec={recall[i]:.4f}, f1={f1[i]:.4f},
supp={support[i]}")

    print(f"\n 混淆矩阵 (行:真实, 列:预测):")
    print("              " + " ".join(f"{name:>6s}" for name in CLASS_NAMES))
    for i, name in enumerate(CLASS_NAMES):
        row_str = " ".join(f"{cm[i,j]:6d}" for j in range(4))
        print(f"   {name:>6s}: {row_str}")

    # 可选的详细分类报告
    print("\n 详细分类报告 (sklearn):")
    print(classification_report(y_true_valid, y_pred_valid, target_names=CLASS_NAMES,
zero_division=0))

def main():
    parser = argparse.ArgumentParser(description="评估任务三：自动定位+液体含量分级 (4 类)")
    parser.add_argument("--dir", required=True, help="测试图片目录")
    parser.add_argument("--label_dir", help="YOLO 标注目录（默认同--dir）")
    parser.add_argument("--model", required=True, help="四分类 YOLO 模型路径 (.pt)")
    parser.add_argument("--conf", type=float, default=0.25, help="检测置信度阈值")
    args = parser.parse_args()

    label_dir = args.label_dir if args.label_dir else args.dir
    print(f"加载模型: {args.model}")
    model = YOLO(args.model)

    evaluate_4class(model, args.dir, label_dir, args.conf)

if __name__ == "__main__":
    main()
```